

SIMULACRO PARCIAL PRACTICO

APLICACIÓN WEB EN JSP

Sistema de pedidos para un restaurante de comidas rápidas

JSP + JSPF + JDBC + MySQL 8 + Bootstrap 5

Elemento	Detalle
Asignatura	Programación Web con Java (Java EE / Jakarta EE)
Tipo de práctica	Taller guiado paso a paso, proyecto completo
Arquitectura	Modelo 1: páginas JSP + fragmentos reutilizables .jspf
Base de datos	MySQL 8 - esquema restaurante_fastfood (10 tablas)
Conexión	Archivo conexion.jspf con JDBC y PreparedStatement
Interfaz	Bootstrap 5.3 desde CDN, diseño responsivo
Servidor	Apache Tomcat 9 (se indica el ajuste para Tomcat 10/11)
Archivos del proyecto	18 archivos: 13 páginas JSP, 5 fragmentos JSPF, CSS y web.xml
Duración estimada	2 sesiones de 2 horas en clase 4 horas en casa.
Estudiante	_____

CONTENIDO

- 1. Presentación del taller 4
 - 1.1 Qué vamos a construir 4
 - 1.2 Competencias que se practican 4
 - 1.3 Entregables 4
 - 1.4 Plan de trabajo por sesiones 4
- 2. Herramientas y requisitos 5
- 3. La base de datos del proyecto 6
 - 3.1 Las tres restricciones UNIQUE que se notan en la aplicación 6
 - 3.2 Modelo entidad-relación 7
- 4. Arquitectura de la aplicación 8
 - 4.1 Qué es un archivo .jspxf y por qué se usa 8
 - 4.2 Archivos del proyecto 9
 - 4.3 Recorrido de una petición 9
- Paso 1. Crear el proyecto y la estructura de carpetas 10
- Paso 2. Instalar el driver JDBC de MySQL 11
- Paso 3. Crear la base de datos y los usuarios de prueba 11
 - Paso 3.1 Usuarios con los que se inicia sesión 11
 - Paso 3.2 Verificación 12
- Paso 4. El archivo de conexión: conexion.jsxpf 13
 - Paso 4.1 Explicación parte por parte 13
 - Paso 4.2 La URL de conexión, parámetro por parámetro 14
 - Paso 4.3 Probar la conexión antes de seguir 14
- Paso 5. Métodos de apoyo: utilidades.jsxpf 16
 - Paso 5.1 Para qué sirve cada método 17
- Paso 6. La plantilla visual con Bootstrap 5 18
 - Paso 6.1 cabecera.jsxpf 18
 - Paso 6.2 Qué aporta Bootstrap 5 aquí 19
 - Paso 6.3 pie.jsxpf 19
 - Paso 6.4 estilos.css 19
- Paso 7. Control de acceso: seguridad.jsxpf 21
 - Paso 7.1 Cómo se usa en cada página 21
- Paso 8. Inicio y cierre de sesión 22
 - Paso 8.1 login.jsxpf 22
 - Paso 8.2 acceso.jsxpf: la validación 23
 - Paso 8.3 Los cuatro puntos clave de la validación 24
 - Paso 8.4 logout.jsxpf 24
- Paso 9. El panel principal: inicio.jsxpf 25
 - Paso 9.1 Detalles que vale la pena mirar 27

Paso 10. Módulo del mesero	28
Paso 10.1 pedidos.jsp: la lista del día	28
Paso 10.2 nuevo.jsp: abrir el pedido	29
Paso 10.3 pedido.jsp: la pantalla de trabajo	30
Paso 10.4 acciones.jsp: el controlador del mesero	35
Paso 11. Módulo de cocina	40
Paso 11.1 tablero.jsp.....	40
Paso 11.2 accion.jsp	42
Paso 12. Módulo de caja	44
Paso 12.1 cobrar.jsp	44
Paso 12.2 facturar.jsp	45
Paso 13. Módulo de administración	49
Paso 13.1 productos.jsp	49
Paso 13.2 guardar_producto.jsp.....	50
Paso 14. Configuración final del proyecto	52
Paso 14.1 index.jsp.....	52
Paso 14.2 web.xml	52
Paso 14.3 Si usa Tomcat 10 u 11	52
Paso 15. Desplegar y probar la aplicación	54
Paso 15.1 Puesta en marcha	54
Paso 15.2 Guion de prueba completo	54
Paso 15.3 Comprobación en la base de datos.....	54
16. Errores frecuentes y cómo resolverlos	56
17. Evaluación y retos	¡Error! Marcador no definido.
17.1 Rúbrica sugerida.....	¡Error! Marcador no definido.
17.2 Retos para subir nota	¡Error! Marcador no definido.
Anexo A. Inventario de archivos entregados	56

Nota: Haga clic derecho sobre la tabla de contenido y elija "Actualizar campos" para ver los números de página.

1. Presentación del taller

1.1 Qué vamos a construir

Durante este taller se desarrolla, de principio a fin, la aplicación web con la que un restaurante de comidas rápidas gestiona sus pedidos. El mesero abre el pedido en una mesa, agrega productos de la carta y lo confirma; en ese momento la comanda aparece en la pantalla de la cocina, donde se toma, se prepara y se marca como lista; el mesero la sirve y finalmente en caja se factura, se registra el pago y la mesa queda libre otra vez.

Todo el proyecto se escribe con páginas JSP y fragmentos .jspf. La conexión a MySQL se centraliza en un único archivo, conexion.jspf, y la presentación se resuelve con Bootstrap 5 para que la aplicación se vea bien tanto en el computador de la caja como en la tablet del mesero.

1.2 Competencias que se practican

- Crear un proyecto web Java y desplegarlo en Apache Tomcat.
- Separar código repetido en fragmentos .jspf e incluirlos con la directiva include.
- Conectar una aplicación web a MySQL con JDBC y el driver mysql-connector-j.
- Escribir consultas seguras con PreparedStatement y evitar la inyección SQL.
- Manejar transacciones (commit y rollback) cuando una operación toca varias tablas.
- Controlar el acceso con sesiones HTTP y perfiles de usuario (mesero, cocina, caja, administrador).
- Construir interfaces responsivas con el sistema de rejilla y los componentes de Bootstrap 5.
- Traducir los errores de las restricciones UNIQUE de la base de datos a mensajes claros para el usuario.

1.3 Entregables

#	Entregable	Descripción
1	Script de la base de datos	Archivo .sql ejecutado en MySQL con las 10 tablas, los datos iniciales y los usuarios de prueba.
2	Proyecto web	Carpeta del proyecto con las 13 páginas JSP, los 5 fragmentos .jspf, el CSS y el web.xml.
3	Evidencia de funcionamiento	Capturas de pantalla del recorrido completo: login, pedido, cocina, servido y factura.
4	Informe corto	Una página explicando qué hace cada fragmento .jspf y por qué se usó una transacción al facturar.

1.4 Plan de trabajo por sesiones

Sesión	Pasos	Resultado esperado al terminar
Sesión 1	Pasos 1 a 5	Proyecto creado, base de datos funcionando y conexión probada desde el navegador.
Sesión 2	Pasos 6 a 10	Login con roles, plantilla Bootstrap y módulo del mesero operativo.
Sesión 3	Pasos 11 a 15	Cocina, caja, administración y la prueba completa de principio a fin.

2. Herramientas y requisitos

Antes de empezar verifique que tiene instalado y funcionando lo siguiente. La versión concreta no es crítica, pero sí la combinación entre Tomcat y el espacio de nombres de Java (javax o jakarta), que es el error más frecuente en clase.

Herramienta	Versión sugerida	Para qué se usa	Cómo verificar
JDK	Java 17 o 26	Compilar las páginas JSP.	java -version en la consola
Apache Tomcat	9.0.x	Servidor de aplicaciones que ejecuta el proyecto.	Abrir http://localhost:8080
MySQL Server	8.0.x	Motor de la base de datos.	mysql -u root -p
MySQL PHPMysqlAdmin	8.0	Ejecutar el script y revisar las tablas.	Conexión local a 127.0.0.1:3306
Driver JDBC	mysql-connector-j-8.3.0.jar	Permite que Java hable con MySQL.	Debe quedar en WEB-INF/lib
IDE	VisualStudioCode	Editar el proyecto y desplegar en Tomcat.	Crear un proyecto web de prueba
Navegador	Chrome, Edge o Firefox	Probar la aplicación.	F12 para ver errores
Bootstrap 5.3	Desde CDN (no se descarga)	Estilos y componentes de la interfaz.	Requiere internet la primera vez

Importante: Tomcat 9 usa el paquete javax.servlet y Tomcat 10 u 11 usan jakarta.servlet. Este taller está escrito para Tomcat 9. Si su instalación es Tomcat 10 o superior, el código JSP funciona igual, pero debe cambiar la cabecera del archivo web.xml a la versión 5.0 con el espacio de nombres jakarta (se explica en el paso 14).

Consejo: Si no tiene internet en el laboratorio, descargue bootstrap.min.css y bootstrap.bundle.min.js, guárdelos en la carpeta css y js del proyecto y reemplace los enlaces del CDN por rutas locales. Todo lo demás del taller no cambia.

3. La base de datos del proyecto

La aplicación trabaja sobre el esquema restaurante_fastfood, diseñado en el documento anterior. Conviene tener presente qué guarda cada tabla, porque cada módulo de la aplicación consulta o actualiza una parte concreta del modelo.

Tabla	Qué guarda	Módulo de la aplicación que la usa
rol	Los cuatro perfiles: administrador, mesero, cocina y cajero.	login (acceso.jsp)
usuario	Empleados con su username y la clave cifrada.	login y control de sesión
mesa	Mesas del local y su estado (libre, ocupada).	mesero: abrir y cerrar pedidos
categoria	Secciones de la carta.	carta del mesero y administración
producto	Ítems del menú con precio y disponibilidad.	carta, cocina y administración
cliente	Datos para domicilios (opcional).	no se usa en esta versión del taller
pedido	Encabezado del pedido y su estado actual.	todos los módulos
detalle_pedido	Líneas del pedido: producto, cantidad y nota.	mesero y cocina
seguimiento_pedido	Bitácora de cambios de estado.	se escribe en cada acción
factura	Cobro del pedido, método de pago y cambio.	caja

3.1 Las tres restricciones UNIQUE que se notan en la aplicación

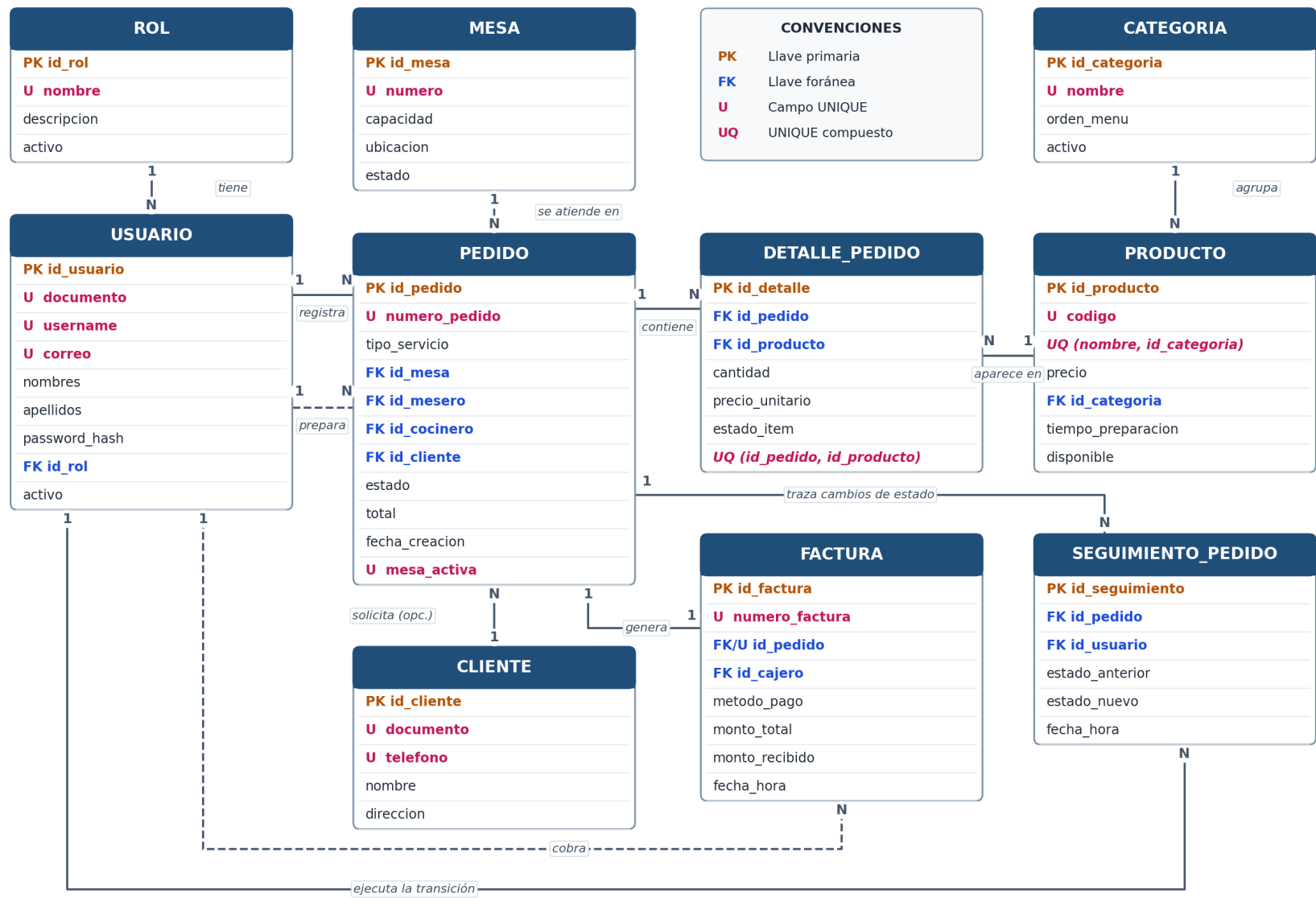
El diseño de la base de datos no es decorativo: tres restricciones aparecen directamente en el comportamiento del programa y en los mensajes que ve el usuario.

Restricción	Qué impide	Dónde se nota en el código
uq_pedido_mesa_activa	Dos pedidos abiertos en la misma mesa.	mesero/acciones.jsp muestra "Esa mesa ya tiene un pedido abierto".
uq_detalle_producto (id_pedido, id_producto)	Que el mismo producto aparezca dos veces en un pedido.	Permite usar INSERT ... ON DUPLICATE KEY UPDATE para sumar la cantidad.
uq_factura_pedido	Facturar dos veces el mismo pedido.	caja/facturar.jsp muestra "Ese pedido ya tiene factura".

El diagrama de la página siguiente resume el modelo completo. Téngalo a la mano mientras escribe las consultas SQL de cada paso.

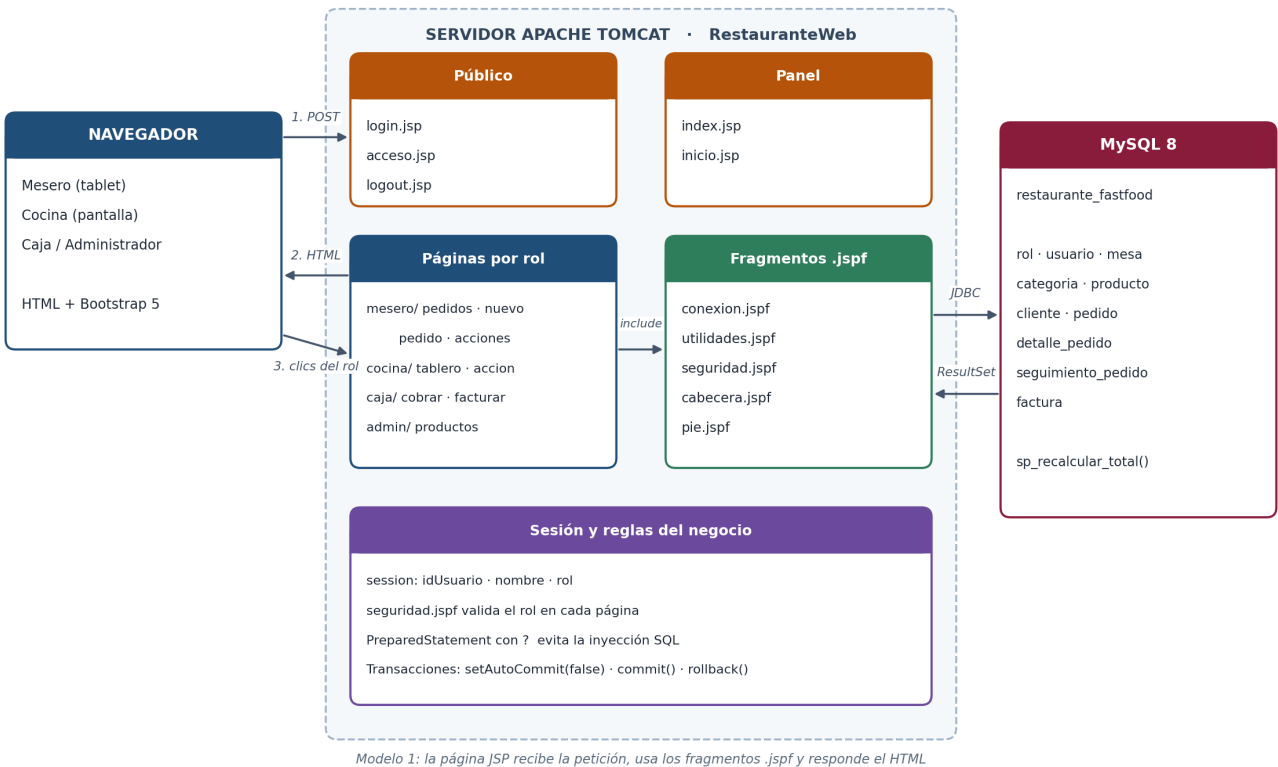
3.2 Modelo entidad-relación

Modelo Entidad-Relación | Sistema de pedidos de un restaurante de comidas rápidas



4. Arquitectura de la aplicación

El proyecto usa el llamado Modelo 1: la propia página JSP recibe la petición del navegador, consulta la base de datos y devuelve el HTML. No se escriben servlets ni clases Java aparte, que es justo lo que pide un primer curso de programación web; el código repetido se saca a fragmentos .jspx y así se conserva el orden.



Modelo 1: la página JSP recibe la petición, usa los fragmentos .jspx y responde el HTML

4.1 Qué es un archivo .jspx y por qué se usa

Un .jspx (JSP Fragment) es un pedazo de página que no se ejecuta por sí solo: se pega dentro de otra página en el momento de compilar, mediante la directiva de inclusión estática:

```
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
```

Como la unión ocurre antes de compilar, la página final es una sola clase Java. Eso tiene dos consecuencias prácticas que hay que entender bien:

- Las variables y los métodos declarados en el fragmento quedan disponibles en la página que lo incluye (por eso conexion.jspf puede ofrecer el método abrirConexion()).
- Un mismo fragmento no se puede incluir dos veces en la misma página: se duplicarían las declaraciones y el compilador daría error.

Forma de incluir	Sintaxis	Cuándo ocurre	Uso en este proyecto
Estática (directiva)	<%@ include file="..." %>	Al compilar la página.	Todos los .jspx del taller.
Dinámica (acción)	<jsp:include page="..." />	Cada vez que se ejecuta.	No se usa: no comparte variables.

Importante: Los fragmentos se guardan dentro de /WEB-INF. Todo lo que está en esa carpeta es invisible desde el navegador: nadie puede escribir la dirección del archivo y leer la clave de MySQL.

4.2 Archivos del proyecto

Archivo	Rol	Qué hace
index.jsp	Entrada	Redirige al login o al panel según haya sesión.
login.jsp	Vista	Formulario de acceso con Bootstrap.
acceso.jsp	Controlador	Valida usuario y clave contra la tabla usuario.
logout.jsp	Controlador	Invalida la sesión.
inicio.jsp	Vista	Panel con indicadores del día y accesos por rol.
prueba_conexion.jsp	Diagnóstico	Comprueba que el driver y la conexión funcionan.
mesero/pedidos.jsp	Vista	Lista los pedidos del mesero.
mesero/nuevo.jsp	Vista	Formulario para abrir un pedido y elegir mesa libre.
mesero/pedido.jsp	Vista	Carta, detalle del pedido y botones de estado.
mesero/acciones.jsp	Controlador	Crear, agregar, quitar, confirmar, servir y cancelar.
cocina/tablero.jsp	Vista	Comandas confirmadas y en preparación.
cocina/accion.jsp	Controlador	Tomar la comanda y marcarla lista.
caja/cobrar.jsp	Vista	Pedidos servidos pendientes de pago.
caja/facturar.jsp	Vista + control	Formulario de pago, factura y comprobante.
admin/productos.jsp	Vista	Menú: alta de productos y disponibilidad.
admin/guardar_producto.jsp	Controlador	Inserta el producto y traduce errores UNIQUE.
WEB-INF/jspf/conexion.jspf	Fragmento	Conexión JDBC centralizada.
WEB-INF/jspf/utilidades.jspf	Fragmento	Cifrado, formato de moneda, escape HTML.
WEB-INF/jspf/seguridad.jspf	Fragmento	Valida sesión y rol.
WEB-INF/jspf/cabecera.jspf	Fragmento	HTML inicial y barra de navegación.
WEB-INF/jspf/pie.jspf	Fragmento	Cierre de la página y JS de Bootstrap.
WEB-INF/web.xml	Configuración	Página inicial, tiempo de sesión y codificación.
css/estilos.css	Estilos	Ajustes propios sobre Bootstrap.

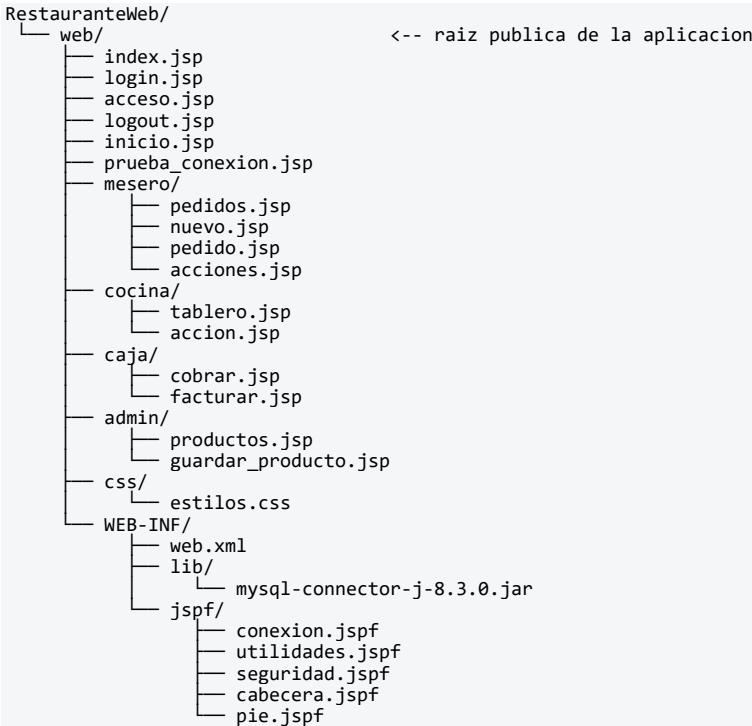
4.3 Recorrido de una petición

Cuando el mesero hace clic en "Confirmar y enviar a cocina" ocurre lo siguiente, en este orden:

- El navegador envía un POST a mesero/acciones.jsp con accion=confirmar y el id del pedido.
- La página incluye conexion.jspf y utilidades.jspf (métodos disponibles) y seguridad.jspf (verifica que hay sesión y que el rol es MESERO).
- Abre una conexión, desactiva el autocommit y ejecuta el UPDATE del estado más el INSERT en seguimiento_pedido.
- Si todo sale bien hace commit; si algo falla hace rollback y ninguna de las dos tablas queda a medias.
- Cierra la conexión y responde con una redirección (sendRedirect) a la lista de pedidos, de modo que al recargar el navegador no se repita la operación.

Paso 1. Crear el proyecto y la estructura de carpetas

Cree un proyecto web llamado RestauranteWeb. La carpeta que en NetBeans se llama "Páginas web", en Eclipse "WebContent" y en un proyecto Maven "src/main/webapp" es la raíz pública: todo lo que aquí se ponga se publica en el servidor. Dentro de ella cree exactamente esta estructura:



Carpeta	Para qué sirve	¿Visible desde el navegador?
raíz (web/)	Páginas que el usuario abre directamente.	Sí
mesero, cocina, caja, admin	Un módulo por rol; facilita el control de acceso.	Sí
css	Hoja de estilos propia.	Sí
WEB-INF	Configuración y fragmentos internos.	No
WEB-INF/lib	Librerías .jar que usa la aplicación.	No
WEB-INF/jspf	Fragmentos reutilizables.	No

Consejo: Trabaje con la extensión .jspx y no .jsp para los fragmentos. Es una convención, pero deja claro a simple vista qué archivos son pedazos de página y cuáles son páginas completas.

Paso 2. Instalar el driver JDBC de MySQL

Java no sabe hablar con MySQL por sí solo: necesita el driver, que es un archivo .jar. Descárguelo del sitio oficial de MySQL (Connector/J, versión 8.3 o superior, opción "Platform Independent") y copie el archivo mysql-connector-j-8.3.0.jar dentro de la carpeta WEB-INF/lib del proyecto.

Entorno	Dónde poner el .jar
VisualStudioCode (proyecto Ant)	Clic derecho en Libraries > Add JAR/Folder, y además copiarlo a WEB-INF/lib.
(Dynamic Web Project)	Arrastrarlo a WebContent/WEB-INF/lib; se agrega solo al classpath.
Maven	Agregar la dependencia com.mysql:mysql-connector-j en el pom.xml.
MySQL	Debe ir dentro del war, en WEB-INF/lib.

Importante: Si el .jar solo está agregado como librería del IDE pero no dentro de WEB-INF/lib, el proyecto compila pero al ejecutarlo aparece el error "No suitable driver found" o "ClassNotFoundException: com.mysql.cj.jdbc.Driver". Es el fallo número uno de este taller.

Paso 3. Crear la base de datos y los usuarios de prueba

Abra MySQL Workbench (o la consola de MySQL) y ejecute el script 01_base_datos_restaurante.sql que acompaña al taller. El script crea el esquema, las diez tablas con sus llaves y restricciones, los índices, las categorías, las mesas, los productos de ejemplo y el procedimiento sp_recalcular_total, que la aplicación usa para actualizar los totales del pedido.

Paso 3.1 Usuarios con los que se inicia sesión

Al final del script se insertan cinco usuarios. La clave de todos es 1234, pero en la tabla no se guarda "1234": se guarda su resumen SHA-256 calculado sobre el texto usuario:clave, exactamente igual que hará después el método claveCifrada() de utilidades.jspf.

```
sql/01_base_datos_restaurante.sql (fragmento)
INSERT INTO usuario (documento, nombres, apellidos, correo, telefono,
                    username, password_hash, id_rol) VALUES
('1098765432', 'Diana', 'Rojas', 'diana.rojas@buensabor.com', '3001112233',
 'admin',
 'f8e68e8d44bfb5314974a97f787d017ff6ac9d0046083f28665fcf96f0cef80c', 1),
('1102233445', 'Carlos', 'Rueda', 'carlos.rueda@buensabor.com', '3002223344',
 'mesero1',
 '6feee5f33d5741297b1d1602d85956b9acece067df55ab7837f43650ec6155a1', 2),
('1104455667', 'Ana', 'Prada', 'ana.prada@buensabor.com', '3004445566',
 'cocina1',
 'd343752b387892e7162a6d1a6f4866ab4d8e989adc16e49fa4a23b3674f2dca3', 3),
('1105566778', 'Luis', 'Ortiz', 'luis.ortiz@buensabor.com', '3005556677',
 'cajero1',
 '8d1bfae7a68bc13df3f5cb20c7e9cac09c2086d5e186c87c46f9bf7a343ef9c9', 4);
```

Usuario	Clave	Rol	Qué puede hacer
admin	1234	ADMINISTRADOR	Ve todos los módulos y administra el menú.
mesero1	1234	MESERO	Toma, confirma y sirve pedidos.
mesero2	1234	MESERO	Segundo mesero, útil para probar la concurrencia.
cocina1	1234	COCINA	Ve el tablero y marca las comandas.
cajero1	1234	CAJERO	Factura los pedidos servidos.

Paso 3.2 Verificación

Ejecute estas dos consultas: la primera debe devolver 10 y la segunda debe listar los cinco usuarios con su rol. Si algo falla, revise que está trabajando sobre el esquema correcto.

```
USE restaurante_fastfood;

SELECT COUNT(*) AS tablas FROM information_schema.tables
WHERE table_schema = 'restaurante_fastfood';

SELECT u.username, r.nombre AS rol FROM usuario u
JOIN rol r ON r.id_rol = u.id_rol ORDER BY u.id_usuario;
```

Importante: Si más adelante quiere agregar un usuario nuevo desde MySQL, no escriba la clave en texto plano. Genere el hash con una página temporal que llame a `claveCifrada("usuario","clave")` y copie el resultado, o use la función de MySQL: `SELECT SHA2('mesero3:1234', 256);`

Paso 4. El archivo de conexión: conexion.jspf

Este es el archivo más importante del taller. Centraliza en un solo lugar los datos de acceso a MySQL y los métodos para abrir y cerrar la conexión. Si mañana el proyecto se lleva a otro computador, solo se cambian cuatro líneas y toda la aplicación sigue funcionando.

```
web/WEB-INF/jspf/conexion.jspf
<!--
  conexion.jspf - Fragmento de conexion a la base de datos MySQL
  Proyecto: RestauranteWeb (Programacion Web con Java)

  Se incluye de forma ESTATICA en cada pagina que necesite la base de datos:
  <%@ include file="/WEB-INF/jspf/conexion.jspf" %>
  Al estar dentro de /WEB-INF ningun usuario puede abrirlo desde el navegador,
  por lo que la clave de MySQL queda protegida.
--%>
<%@ page import="java.sql.*" %>
<%!
  /* ===== PARAMETROS DE CONEXION =====
  Solo hay que modificar estas cuatro constantes para llevar el
  proyecto a otro computador o a otro servidor. */
  public static final String DB_DRIVER = "com.mysql.cj.jdbc.Driver";
  public static final String DB_URL =
    "jdbc:mysql://localhost:3306/restaurante_fastfood"
    + "?useSSL=false&allowPublicKeyRetrieval=true"
    + "&serverTimezone=America/Bogota&characterEncoding=UTF-8";
  public static final String DB_USUARIO = "root";
  public static final String DB_CLAVE = "";

  /**
   * Abre y devuelve una conexion nueva con la base de datos.
   * Quien la llama es responsable de cerrarla (preferiblemente con cerrar()).
   */
  public Connection abrirConexion() throws SQLException {
    try {
      Class.forName(DB_DRIVER);          // carga el driver JDBC
    } catch (ClassNotFoundException ex) {
      throw new SQLException(
        "No se encontro el driver de MySQL. Agregue mysql-connector-j-8.x.jar "
        + "a la carpeta WEB-INF/lib del proyecto.", ex);
    }
    return DriverManager.getConnection(DB_URL, DB_USUARIO, DB_CLAVE);
  }

  /**
   * Cierra en orden inverso cualquier recurso JDBC (ResultSet, Statement,
   * Connection) sin obligar a escribir try/catch en cada pagina.
   */
  public void cerrar(AutoCloseable... recursos) {
    for (int i = recursos.length - 1; i >= 0; i--) {
      if (recursos[i] != null) {
        try { recursos[i].close(); } catch (Exception ignorada) { }
      }
    }
  }

  /** Deshace la transaccion en curso sin propagar errores secundarios. */
  public void deshacer(Connection con) {
    if (con != null) {
      try { con.rollback(); } catch (SQLException ignorada) { }
    }
  }
}
%>
```

Paso 4.1 Explicación parte por parte

Elemento	Qué hace y por qué está ahí
<!-- ... --%>	Comentario JSP: no se envía al navegador, a diferencia del comentario HTML <!-- -->.
<%@ page import="java.sql.*" %>	Importa Connection, PreparedStatement, ResultSet y SQLException para todas las páginas que incluyan el fragmento.
<%! ... %>	Bloque de declaración: lo que se escribe aquí se convierte en atributos y métodos de la clase del servlet, no en código del método de servicio. Por eso pueden declararse constantes y métodos.
DB_DRIVER	Nombre de la clase del driver. En el conector 8 es com.mysql.cj.jdbc.Driver (con .cj).

DB_URL	Dirección de la base de datos con sus parámetros.
DB_USUARIO / DB_CLAVE	Credenciales de MySQL. Cámbielas por las de su equipo.
abrirConexion()	Carga el driver con Class.forName y devuelve una conexión nueva con DriverManager.getConnection.
cerrar(AutoCloseable...)	Recibe cualquier cantidad de recursos y los cierra en orden inverso, ignorando los errores del cierre. Evita repetir bloques finally enormes.
deshacer(Connection)	Ejecuta rollback() de forma segura cuando una transacción falla.

Paso 4.2 La URL de conexión, parámetro por parámetro

Parte de la URL	Significado
jdbc:mysql://	Protocolo JDBC y motor al que se conecta.
localhost:3306	Servidor y puerto de MySQL. Si el motor está en otro equipo, aquí va su IP.
/restaurante_fastfood	Nombre del esquema creado en el paso 3.
useSSL=false	Evita la advertencia de conexión no cifrada en entornos locales.
allowPublicKeyRetrieval=true	Necesario con la autenticación por defecto de MySQL 8.
serverTimezone=America/Bogota	Sin este parámetro MySQL 8 puede lanzar un error de zona horaria.
characterEncoding=UTF-8	Permite guardar tildes y la letra ñ correctamente.

Paso 4.3 Probar la conexión antes de seguir

No avance al paso 5 sin comprobar que la conexión funciona. Cree esta página temporal en la raíz del proyecto, ejecútela y bórrala cuando termine el taller.

```
web/prueba_conexion.jsp
<!--
prueba_conexion.jsp - Pagina temporal para comprobar que el driver JDBC
y los datos de conexion.jspf funcionan. BORRELA al terminar el taller.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8"><title>Prueba de conexion</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="p-5">
<div class="container" style="max-width:700px">
<%
    Connection con = null; Statement st = null; ResultSet rs = null;
    try {
        con = abrirConexion();
        st = con.createStatement();
        rs = st.executeQuery("SELECT COUNT(*) AS total FROM producto");
        rs.next();
    %>
    <div class="alert alert-success">
        <h4>Conexion exitosa</h4>
        Base de datos: <b><%= con.getCatalog() %></b><br>
        Motor: <%= con.getMetaData().getDatabaseProductVersion() %><br>
        Productos registrados: <b><%= rs.getInt("total") %></b>
    </div>
    <%
    } catch (SQLException ex) {
    %>
    <div class="alert alert-danger">
        <h4>Fallo la conexion</h4>
        <pre class="mb-0"><%= ex.getMessage() %></pre>
    </div>
    <%
    } finally { cerrar(rs, st, con); }
    %>
</div>
</body>
</html>
```

Abra http://localhost:8080/RestauranteWeb/prueba_conexion.jsp. Si aparece el aviso verde con el número de productos, la conexión quedó lista. Si sale el aviso rojo, compare el mensaje con la tabla de errores del capítulo final.

Paso 5. Métodos de apoyo: utilidades.jspf

El segundo fragmento reúne pequeñas funciones que se repetirían en todas las páginas: cifrar la clave, mostrar los precios en pesos, convertir los parámetros del formulario a número y escapar el texto que escribe el usuario.

web/WEB-INF/jspf/utilidades.jspf

```
<!--
    utilidades.jspf - Metodos auxiliares reutilizables en todo el proyecto.
    Se incluye siempre DESPUES de conexion.jspf.
-->
<%@ page import="java.security.MessageDigest, java.text.NumberFormat, java.util.Locale" %>
<%!

/** Cifra un texto con SHA-256 y lo devuelve en hexadecimal.
 * La clave del usuario NUNCA se guarda en texto plano.
 * En produccion se recomienda BCrypt; aqui se usa SHA-256 para no
 * depender de librerias externas. */
public String sha256(String texto) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] resumen = md.digest(texto.getBytes("UTF-8"));
        StringBuilder sb = new StringBuilder();
        for (byte b : resumen) sb.append(String.format("%02x", b));
        return sb.toString();
    } catch (Exception ex) {
        throw new RuntimeException("Error al cifrar la clave", ex);
    }
}

/** Combina usuario y clave antes de cifrar: dos usuarios con la misma
 * clave producen hashes distintos (salt sencillo). */
public String claveCifrada(String usuario, String clave) {
    return sha256(usuario + ":" + clave);
}

/** Formatea un valor como moneda colombiana: 15000 -> $ 15.000 */
public String pesos(double valor) {
    NumberFormat f = NumberFormat.getInstance(new Locale("es", "CO"));
    f.setMaximumFractionDigits(0);
    return "$ " + f.format(valor);
}

/** Convierte a entero un parametro del formulario; si viene vacio o
 * con basura devuelve el valor por defecto (evita NumberFormatException). */
public int aEntero(String valor, int porDefecto) {
    try { return Integer.parseInt(valor.trim()); }
    catch (Exception ex) { return porDefecto; }
}

public double aDoble(String valor, double porDefecto) {
    try { return Double.parseDouble(valor.trim().replace(",", ".")); }
    catch (Exception ex) { return porDefecto; }
}

/** Escapa caracteres HTML para evitar ataques XSS al mostrar datos
 * escritos por el usuario (por ejemplo la nota de un producto). */
public String esc(String texto) {
    if (texto == null) return "";
    return texto.replace("&", "&amp;").replace("<", "&lt;")
        .replace(">", "&gt;").replace("\\"", "&quot;");
}

/** Devuelve el color de Bootstrap que corresponde a cada estado. */
public String colorEstado(String estado) {
    if (estado == null) return "secondary";
    switch (estado) {
        case "PENDIENTE": return "secondary";
        case "CONFIRMADO": return "warning";
        case "EN_PREPARACION": return "info";
        case "LISTO": return "success";
        case "SERVIDO": return "primary";
        case "PAGADO": return "dark";
        case "CANCELADO": return "danger";
        default: return "secondary";
    }
}
%>
```

Paso 5.1 Para qué sirve cada método

Método	Uso en la aplicación
<code>sha256(texto)</code>	Calcula el resumen criptográfico. Es de una sola vía: del hash no se puede volver a la clave.
<code>claveCifrada(usuario, clave)</code>	Une el usuario y la clave antes de cifrar. Así dos empleados con la clave 1234 tienen hashes distintos.
<code>pesos(valor)</code>	Convierte 15000 en "\$ 15.000". Se usa en todas las tablas de precios.
<code>aEntero(valor, defecto)</code>	Convierte un parámetro de texto en entero sin que la página se caiga si viene vacío o alterado.
<code>aDoble(valor, defecto)</code>	Igual que el anterior para valores con decimales, como el dinero recibido.
<code>esc(texto)</code>	Reemplaza < > & " por sus entidades HTML. Impide que alguien escriba una etiqueta script en la nota de un producto (ataque XSS).
<code>colorEstado(estado)</code>	Devuelve el color de Bootstrap de cada estado del pedido, para pintar las etiquetas de colores.

Importante: SHA-256 es suficiente para un ejercicio académico, pero en un sistema real se usa BCrypt o Argon2, que son lentos a propósito para dificultar los ataques por fuerza bruta. Menciónelo en el informe: demuestra criterio.

Paso 6. La plantilla visual con Bootstrap 5

Toda página del sistema tiene la misma estructura: encabezado HTML con Bootstrap, barra de navegación con las opciones del rol, contenido, y pie. En lugar de repetir ese HTML dieciséis veces se parte en dos fragmentos: cabecera.jspf y pie.jspf. Cada página queda así:

```
<!-- 1. directivas y consultas de la pagina --%>
<% String tituloPagina = "Mis pedidos"; %>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

    ... aqui va el contenido propio de la pagina ...

<%@ include file="/WEB-INF/jspf/pie.jspf" %>
```

Paso 6.1 cabecera.jspf

```
web/WEB-INF/jspf/cabecera.jspf
<!--
    cabecera.jspf - Encabezado HTML + barra de navegacion Bootstrap 5.
    La pagina debe definir antes la variable tituloPagina.
--%>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title><%= tituloPagina %> | Restaurante El Buen Sabor</title>

    <!-- Bootstrap 5.3 desde CDN: hoja de estilos -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
        rel="stylesheet">
    <!-- Iconos de Bootstrap -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.css"
        rel="stylesheet">
    <link href="<%= ctx %>/css/estilos.css" rel="stylesheet">
</head>
<body class="bg-light">

<nav class="navbar navbar-expand-lg navbar-dark bg-dark shadow-sm">
    <div class="container">
        <a class="navbar-brand fw-bold" href="<%= ctx %>/inicio.jsp">
            <i class="bi bi-shop"></i> El Buen Sabor
        </a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
            data-bs-target="#menu">
            <span class="navbar-toggler-icon"></span>
        </button>

        <div class="collapse navbar-collapse" id="menu">
            <ul class="navbar-nav me-auto">
                <li class="nav-item">
                    <a class="nav-link" href="<%= ctx %>/inicio.jsp">
                        <i class="bi bi-house"></i> Inicio</a></li>

                <% if ("MESERO".equals(rolSesion) || "ADMINISTRADOR".equals(rolSesion)) { %>
                <li class="nav-item">
                    <a class="nav-link" href="<%= ctx %>/mesero/pedidos.jsp">
                        <i class="bi bi-receipt"></i> Pedidos</a></li>
                <% } %>

                <% if ("COCINA".equals(rolSesion) || "ADMINISTRADOR".equals(rolSesion)) { %>
                <li class="nav-item">
                    <a class="nav-link" href="<%= ctx %>/cocina/tablero.jsp">
                        <i class="bi bi-fire"></i> Cocina</a></li>
                <% } %>

                <% if ("CAJERO".equals(rolSesion) || "MESERO".equals(rolSesion)
                    || "ADMINISTRADOR".equals(rolSesion)) { %>
                <li class="nav-item">
                    <a class="nav-link" href="<%= ctx %>/caja/cobrar.jsp">
                        <i class="bi bi-cash-coin"></i> Caja</a></li>
                <% } %>

                <% if ("ADMINISTRADOR".equals(rolSesion)) { %>
                <li class="nav-item">
                    <a class="nav-link" href="<%= ctx %>/admin/productos.jsp">
                        <i class="bi bi-card-list"></i> Productos</a></li>
```

```

        <% } %>
    </ul>

    <span class="navbar-text text-white me-3">
        <i class="bi bi-person-circle"></i> <%= esc(nombreSesion) %>
        <span class="badge text-bg-warning ms-1"><%= rolSesion %></span>
    </span>
    <a class="btn btn-outline-light btn-sm" href="<%= ctx %>/logout.jsp">
        <i class="bi bi-box-arrow-right"></i> Salir</a>
    </div>
</div>
</nav>

<main class="container my-4">

```

Paso 6.2 Qué aporta Bootstrap 5 aquí

Clase o componente	Efecto visual
navbar navbar-expand-lg navbar-dark bg-dark	Barra de navegación oscura que se colapsa en un botón cuando la pantalla es pequeña.
container	Centra el contenido y le da márgenes automáticos según el ancho de la pantalla.
collapse navbar-collapse + data-bs-toggle	Menú desplegable en móvil; funciona con el JavaScript que carga pie.jspf.
badge text-bg-warning	Etiqueta de color; se usa para el rol y para el estado del pedido.
btn btn-outline-light btn-sm	Botón de borde claro y tamaño reducido.
bi bi-*	Iconos de Bootstrap Icons: bi-shop, bi-fire, bi-cash-coin, etc.

La barra de navegación no es igual para todos: los enlaces se muestran según el rol guardado en la sesión. Un cocinero no ve el módulo de administración porque el bloque correspondiente no se imprime.

Paso 6.3 pie.jspf

```

web/WEB-INF/jspf/pie.jspf
<%-- pie.jspf - Cierre de la pagina y carga del JavaScript de Bootstrap. --%>
</main>

<footer class="text-center text-muted py-3 border-top bg-white">
    <small>Restaurante El Buen Sabor &middot; Sistema de pedidos JSP + MySQL &middot;
    Programacion Web Java</small>
</footer>

<!-- Bootstrap 5.3: JavaScript (menu responsivo, modales, alertas) -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</script>
</body>
</html>

```

Importante: El archivo bootstrap.bundle.min.js debe cargarse al final del documento, no en la cabecera. Si se carga antes que el HTML, los menús y las alertas descartables no responden.

Paso 6.4 estilos.css

```

web/css/estilos.css
/* estilos.css - ajustes propios sobre Bootstrap 5 */

body { font-family: "Segoe UI", Roboto, Arial, sans-serif; }

/* Tarjeta de comanda en el tablero de cocina */
.comanda { border-left: 6px solid #0dcaf0; }
.comanda.espera { border-left-color: #ffc107; }
.comanda.tarde { border-left-color: #dc3545; }

/* Producto de la carta */
.producto:hover { background-color: #fff8e6; cursor: pointer; }

/* Impresion del comprobante: oculta menus y botones */
@media print {
    nav, footer, .no-imprimir { display: none !important; }
    .card { border: none !important; }
}

```

Son solo cuatro ajustes: el borde de color de las comandas de cocina, el resaltado de los productos de la carta y una regla de impresión que oculta los menús cuando se imprime el comprobante de la factura.

Paso 7. Control de acceso: seguridad.jspf

Sin este fragmento cualquiera podría escribir la dirección cocina/tablero.jsp y entrar sin iniciar sesión. La idea es sencilla: cada página declara qué roles pueden verla y el fragmento se encarga de comprobarlo.

```
web/WEB-INF/jspf/seguridad.jspf
<!--
seguridad.jspf - Control de acceso por sesion y por rol.

Cada pagina protegida declara ANTES del include el arreglo con los roles
autorizados y luego incluye este fragmento:

    <% String[] rolesPermitidos = {"MESERO", "ADMINISTRADOR"}; %>
    <%@ include file="/WEB-INF/jspf/seguridad.jspf" %>

Como el include es estatico, la variable rolesPermitidos es visible aqui.
-->
<%
// Datos guardados en la sesion cuando el usuario inicio sesion
Integer idUsuarioSesion = (Integer) session.getAttribute("idUsuario");
String rolSesion = (String) session.getAttribute("rol");
String nombreSesion = (String) session.getAttribute("nombre");
String ctx = request.getContextPath();

// 1) Si no hay sesion activa, regresa al login
if (idUsuarioSesion == null) {
    response.sendRedirect(ctx + "/login.jsp?error=sesion");
    return;
}

// 2) Si el rol del usuario no esta autorizado en esta pagina, lo bloquea
boolean autorizado = false;
for (String r : rolesPermitidos) {
    if (r.equals(rolSesion)) { autorizado = true; break; }
}
if (!autorizado) {
    response.sendRedirect(ctx + "/inicio.jsp?error=permiso");
    return;
}
%>
```

Paso 7.1 Cómo se usa en cada página

```
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"MESERO", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
```

El orden importa. La variable rolesPermitidos se declara antes del include porque, al ser una inclusión estática, el fragmento se pega justo debajo y necesita que la variable ya exista.

Línea del fragmento	Explicación
<code>session.getAttribute("idUsuario")</code>	Recupera el identificador guardado al iniciar sesión. Si es null, no hay sesión activa.
<code>request.getContextPath()</code>	Devuelve el nombre de la aplicación (/RestauranteWeb). Se guarda en la variable ctx y se usa en todos los enlaces para que funcionen desde cualquier subcarpeta.
<code>response.sendRedirect(...)</code>	Envía al navegador a otra dirección.
<code>return;</code>	Corta la ejecución de la página. Sin este return, el resto del HTML se seguiría generando después de la redirección y saldría el error "response already committed".
<code>for (String r : rolesPermitidos)</code>	Recorre los roles autorizados y compara con el de la sesión.

Consejo: Los roles se escriben exactamente como están en la tabla rol: ADMINISTRADOR, MESERO, COCINA y CAJERO. Una diferencia de mayúsculas hace que el usuario quede bloqueado en su propio módulo.

Paso 8. Inicio y cierre de sesión

El acceso se resuelve con tres páginas: el formulario (login.jsp), el validador (acceso.jsp) y la salida (logout.jsp). Ninguna de las tres incluye seguridad.jspf, porque son justamente las páginas públicas.

Paso 8.1 login.jsp

```
web/login.jsp
<!--
login.jsp - Formulario de inicio de sesion.
Es la unica pagina publica del sistema: no incluye seguridad.jspf.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%
String ctx = request.getContextPath();
String error = request.getParameter("error");
String mensaje = null;
if ("clave".equals(error)) mensaje = "Usuario o clave incorrectos.";
if ("sesion".equals(error)) mensaje = "Debe iniciar sesion para continuar.";
if ("vacio".equals(error)) mensaje = "Escriba el usuario y la clave.";
if ("inactivo".equals(error)) mensaje = "El usuario esta inhabilitado.";
%>
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Iniciar sesion | El Buen Sabor</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.css"
rel="stylesheet">
<link href="<%= ctx %>/css/estilos.css" rel="stylesheet">
</head>
<body class="bg-dark d-flex align-items-center" style="min-height:100vh">

<div class="container" style="max-width:420px">
<div class="card shadow-lg border-0">
<div class="card-body p-4">

<div class="text-center mb-4">
<i class="bi bi-shop display-4 text-warning"></i>
<h4 class="mt-2 mb-0 fw-bold">El Buen Sabor</h4>
<small class="text-muted">Sistema de pedidos</small>
</div>

<% if (mensaje != null) { %>
<div class="alert alert-danger py-2">
<i class="bi bi-exclamation-triangle"></i> <%= mensaje %>
</div>
<% } %>

<!-- El formulario envia los datos por POST a acceso.jsp -->
<form method="post" action="<%= ctx %>/acceso.jsp">
<div class="mb-3">
<label class="form-label" for="usuario">Usuario</label>
<div class="input-group">
<span class="input-group-text"><i class="bi bi-person"></i></span>
<input type="text" class="form-control" id="usuario" name="usuario"
required autofocus placeholder="mesero1">
</div>
</div>

<div class="mb-4">
<label class="form-label" for="clave">Clave</label>
<div class="input-group">
<span class="input-group-text"><i class="bi bi-key"></i></span>
<input type="password" class="form-control" id="clave" name="clave"
required placeholder="*****">
</div>
</div>

<button type="submit" class="btn btn-warning w-100 fw-bold">
<i class="bi bi-box-arrow-in-right"></i> Ingresar
</button>
</form>

<hr class="my-4">
```

```

        <p class="small text-muted mb-1"><b>Usuarios de prueba</b> (clave: 1234)</p>
        <ul class="small text-muted mb-0">
            <li>admin &middot; mesero1 &middot; cocina1 &middot; cajero1</li>
        </ul>
    </div>
</div>
</div>
</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</script>
</body>
</html>

```

La página lee el parámetro error de la dirección para mostrar el mensaje adecuado. Ese parámetro lo agrega acceso.jsp o seguridad.jspf cuando algo falla; así se evita usar variables de sesión para mensajes de un solo uso.

Clase de Bootstrap	Para qué se usó
d-flex align-items-center + min-height:100vh	Centra la tarjeta verticalmente en la pantalla.
card shadow-lg	Tarjeta blanca con sombra sobre el fondo oscuro.
input-group + input-group-text	Campo de texto con un icono pegado a la izquierda.
alert alert-danger	Franja roja con el mensaje de error.
btn btn-warning w-100	Botón amarillo que ocupa todo el ancho disponible.

Paso 8.2 acceso.jsp: la validación

```

web/acceso.jsp
<!--
    acceso.jsp - Valida las credenciales contra la tabla usuario.
    No genera HTML: solo procesa y redirige.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<%
    request.setCharacterEncoding("UTF-8");
    String ctx = request.getContextPath();
    String usuario = request.getParameter("usuario");
    String clave = request.getParameter("clave");

    if (usuario == null || clave == null
        || usuario.trim().isEmpty() || clave.trim().isEmpty()) {
        response.sendRedirect(ctx + "/login.jsp?error=vacio");
        return;
    }
    usuario = usuario.trim();

    Connection con = null;
    PreparedStatement ps = null;
    ResultSet rs = null;
    try {
        con = abrirConexion();

        // Consulta con PreparedStatement: los ? evitan la inyeccion SQL
        String sql = "SELECT u.id_usuario, u.nombres, u.apellidos, "
            + "      u.password_hash, u.activo, r.nombre AS rol "
            + "FROM usuario u "
            + "      JOIN rol r ON r.id_rol = u.id_rol "
            + "WHERE u.username = ?";
        ps = con.prepareStatement(sql);
        ps.setString(1, usuario);
        rs = ps.executeQuery();

        if (rs.next()) {
            String hashGuardado = rs.getString("password_hash");
            boolean activo = rs.getBoolean("activo");

            if (!activo) {
                response.sendRedirect(ctx + "/login.jsp?error=inactive");
                return;
            }
            // Se cifra lo que escribio el usuario y se compara con la BD
            if (hashGuardado.equals(claveCifrada(usuario, clave))) {

```

```

        session.setAttribute("idUserario", rs.getInt("id_usuario"));
        session.setAttribute("nombre",
            rs.getString("nombres") + " " + rs.getString("apellidos"));
        session.setAttribute("rol", rs.getString("rol"));
        session.setAttribute("username", usuario);
        session.setMaxInactiveInterval(30 * 60); // 30 minutos

        // Deja constancia del ingreso
        PreparedStatement psAcceso = con.prepareStatement(
            "UPDATE usuario SET ultimo_acceso = NOW() WHERE id_usuario = ?");
        psAcceso.setInt(1, rs.getInt("id_usuario"));
        psAcceso.executeUpdate();
        cerrar(psAcceso);

        response.sendRedirect(ctx + "/inicio.jsp");
        return;
    }
}
response.sendRedirect(ctx + "/login.jsp?error=clave");

} catch (SQLException ex) {
    out.println("<div style='font-family:sans-serif;padding:20px'>"
        + "<h3>Error de conexion con la base de datos</h3><pre>"
        + ex.getMessage() + "</pre></div>");
} finally {
    cerrar(rs, ps, con);
}
%>

```

Paso 8.3 Los cuatro puntos clave de la validación

Punto	Explicación
PreparedStatement con ?	La consulta se envía con un hueco y el valor viaja aparte. Si alguien escribe ' OR '1'=1 como usuario, se busca literalmente ese texto y no se altera la consulta. Concatenar el parámetro dentro del SQL sería una vulnerabilidad de inyección.
Comparación de hashes	Nunca se consulta "WHERE clave = ?". Se trae el hash guardado y se compara con el hash de lo que escribió el usuario.
Atributos de sesión	Se guardan idUsuario, nombre, rol y username. La clave jamás se guarda en la sesión.
setMaxInactiveInterval(30*60)	La sesión caduca a los 30 minutos sin actividad, útil en un equipo compartido del restaurante.

Paso 8.4 logout.jsp

```

web/logout.jsp
<!-- logout.jsp - Cierra la sesion y regresa al formulario de acceso. -->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%
    session.invalidate(); // elimina todos los atributos
    response.sendRedirect("login.jsp");
%>

```

invalidate() destruye la sesión completa: todos los atributos desaparecen y el siguiente acceso a cualquier página protegida vuelve al login.

Paso 9. El panel principal: inicio.jsp

Es la primera página que se ve tras iniciar sesión. Muestra cuatro indicadores del día y las tarjetas de los módulos a los que el usuario tiene derecho según su rol.

```
web/inicio.jsp
<!--
  inicio.jsp - Panel principal. Muestra indicadores del día y los accesos
  directos que corresponden al rol del usuario.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"ADMINISTRADOR", "MESERO", "COCINA", "CAJERO"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
  String tituloPagina = "Inicio";

  int abiertos = 0, enCocina = 0, listos = 0;
  double ventaDia = 0;

  Connection con = null; Statement st = null; ResultSet rs = null;
  try {
    con = abrirConexion();
    st = con.createStatement();

    rs = st.executeQuery(
      "SELECT "
      + " SUM(estado IN ('PENDIENTE','CONFIRMADO'))           AS abiertos, "
      + " SUM(estado = 'EN_PREPARACION')                     AS cocina, "
      + " SUM(estado = 'LISTO')                               AS listos "
      + "FROM pedido WHERE DATE(fecha_creacion) = CURDATE());"
    if (rs.next()) {
      abiertos = rs.getInt("abiertos");
      enCocina = rs.getInt("cocina");
      listos = rs.getInt("listos");
    }
    cerrar(rs);

    rs = st.executeQuery(
      "SELECT IFNULL(SUM(monto_total),0) AS venta FROM factura "
      + "WHERE DATE(fecha_hora) = CURDATE() AND anulada = 0");
    if (rs.next()) ventaDia = rs.getDouble("venta");

  } catch (SQLException ex) {
    request.setAttribute("errorBD", ex.getMessage());
  } finally {
    cerrar(rs, st, con);
  }
%>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

<h3 class="mb-1">Hola, <%= esc(nombreSesion) %></h3>
<p class="text-muted">Panel de control del <%= rolSesion.toLowerCase() %>.</p>

<% if ("permiso".equals(request.getParameter("error"))) { %>
  <div class="alert alert-warning">
    <i class="bi bi-shield-lock"></i>
    No tiene permisos para entrar a esa seccion con el rol <%= rolSesion %>.
  </div>
<% } %>
<% if (request.getAttribute("errorBD") != null) { %>
  <div class="alert alert-danger">Error de base de datos:
    <%= esc((String) request.getAttribute("errorBD")) %></div>
<% } %>

<!-- ===== Indicadores del dia ===== -->
<div class="row g-3 mb-4">
  <div class="col-6 col-lg-3">
    <div class="card border-0 shadow-sm text-center h-100">
      <div class="card-body">
        <i class="bi bi-journal-text fs-2 text-secondary"></i>
        <h3 class="mt-2 mb-0"><%= abiertos %></h3>
        <small class="text-muted">Pedidos abiertos</small>
      </div>
    </div>
  </div>
  <div class="col-6 col-lg-3">
    <div class="card border-0 shadow-sm text-center h-100">
```

```

        <div class="card-body">
            <i class="bi bi-fire fs-2 text-info"></i>
            <h3 class="mt-2 mb-0"><%= enCocina %></h3>
            <small class="text-muted">En preparacion</small>
        </div>
    </div>
</div>
<div class="col-6 col-lg-3">
    <div class="card border-0 shadow-sm text-center h-100">
        <div class="card-body">
            <i class="bi bi-bell fs-2 text-success"></i>
            <h3 class="mt-2 mb-0"><%= listos %></h3>
            <small class="text-muted">Listos para servir</small>
        </div>
    </div>
</div>
<div class="col-6 col-lg-3">
    <div class="card border-0 shadow-sm text-center h-100">
        <div class="card-body">
            <i class="bi bi-cash-stack fs-2 text-dark"></i>
            <h3 class="mt-2 mb-0"><%= pesos(ventaDia) %></h3>
            <small class="text-muted">Venta de hoy</small>
        </div>
    </div>
</div>
</div>
</div>

<!-- ===== Accesos directos por rol ===== -->
<div class="row g-3">
    <% if ("MESERO".equals(rolSesion) || "ADMINISTRADOR".equals(rolSesion)) { %>
    <div class="col-md-4">
        <div class="card h-100 shadow-sm">
            <div class="card-body">
                <h5><i class="bi bi-receipt text-warning"></i> Modulo mesero</h5>
                <p class="small text-muted">Tomar pedidos, confirmarlos y servirlos.</p>
                <a href="<%= ctx %>/mesero/pedidos.jsp" class="btn btn-warning btn-sm">Entrar</a>
            </div>
        </div>
    </div>
    <% } %>
    <% if ("COCINA".equals(rolSesion) || "ADMINISTRADOR".equals(rolSesion)) { %>
    <div class="col-md-4">
        <div class="card h-100 shadow-sm">
            <div class="card-body">
                <h5><i class="bi bi-fire text-info"></i> Modulo cocina</h5>
                <p class="small text-muted">Ver comandas confirmadas y marcarlas listas.</p>
                <a href="<%= ctx %>/cocina/tablero.jsp" class="btn btn-info btn-sm">Entrar</a>
            </div>
        </div>
    </div>
    <% } %>
    <% if ("CAJERO".equals(rolSesion) || "MESERO".equals(rolSesion)
        || "ADMINISTRADOR".equals(rolSesion)) { %>
    <div class="col-md-4">
        <div class="card h-100 shadow-sm">
            <div class="card-body">
                <h5><i class="bi bi-cash-coin text-success"></i> Modulo caja</h5>
                <p class="small text-muted">Facturar los pedidos servidos y liberar la mesa.</p>
                <a href="<%= ctx %>/caja/cobrar.jsp" class="btn btn-success btn-sm">Entrar</a>
            </div>
        </div>
    </div>
    <% } %>
    <% if ("ADMINISTRADOR".equals(rolSesion)) { %>
    <div class="col-md-4">
        <div class="card h-100 shadow-sm">
            <div class="card-body">
                <h5><i class="bi bi-card-list text-primary"></i> Administrar menu</h5>
                <p class="small text-muted">Crear productos, cambiar precios y disponibilidad.</p>
                <a href="<%= ctx %>/admin/productos.jsp" class="btn btn-primary btn-sm">Entrar</a>
            </div>
        </div>
    </div>
    <% } %>
</div>
</div>
<@ include file="/WEB-INF/jspf/pie.jspf" %>

```

Paso 9.1 Detalles que vale la pena mirar

Detalle	Explicación
SUM(estado IN ('PENDIENTE', 'CONFIRMADO'))	En MySQL una comparación devuelve 1 o 0, así que sumarla cuenta las filas que cumplen la condición. Con una sola consulta se obtienen tres contadores.
IFNULL(SUM(monto_total),0)	Si todavía no hay facturas, SUM devuelve NULL; IFNULL lo convierte en 0 para que no se muestre vacío.
row g-3 + col-6 col-lg-3	Rejilla de Bootstrap: dos tarjetas por fila en el celular y cuatro en el computador.
Bloques condicionales por rol	Cada tarjeta está envuelta en un if que consulta rolSesion, la variable que aportó seguridad.jspf.
cerrar(rs, st, con) en finally	La conexión se cierra siempre, incluso si la consulta falla. Dejar conexiones abiertas agota el servidor de base de datos.

Paso 10. Módulo del mesero

Es el módulo más extenso porque concentra el trabajo del salón: ver los pedidos propios, abrir uno nuevo, agregarle productos y hacerlo avanzar de estado. Está formado por tres vistas y un controlador.

Paso 10.1 pedidos.jsp: la lista del día

```
web/mesero/pedidos.jsp
<!--
mesero/pedidos.jsp - Lista los pedidos del mesero que inicio sesion.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"MESERO", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    String tituloPagina = "Mis pedidos";
    String msg = request.getParameter("msg");
    String err = request.getParameter("err");
%>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

<div class="d-flex justify-content-between align-items-center mb-3">
    <h3 class="mb-0"><i class="bi bi-receipt"></i> Mis pedidos de hoy</h3>
    <a href="<%= ctx %>/mesero/nuevo.jsp" class="btn btn-warning">
        <i class="bi bi-plus-circle"></i> Nuevo pedido</a>
</div>

<% if (msg != null) { %>
    <div class="alert alert-success alert-dismissible fade show">
        <i class="bi bi-check-circle"></i> <%= esc(msg) %>
        <button class="btn-close" data-bs-dismiss="alert"></button>
    </div>
<% } %>
<% if (err != null) { %>
    <div class="alert alert-danger alert-dismissible fade show">
        <i class="bi bi-x-circle"></i> <%= esc(err) %>
        <button class="btn-close" data-bs-dismiss="alert"></button>
    </div>
<% } %>

<div class="card shadow-sm">
    <div class="table-responsive">
        <table class="table table-hover align-middle mb-0">
            <thead class="table-dark">
                <tr>
                    <th>Pedido</th><th>Mesa</th><th>Tipo</th><th>Estado</th>
                    <th class="text-end">Total</th><th>Hora</th><th></th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td class="fw-semibold"><%= rs.getString("numero_pedido") %></td>
                    <td><%= rs.getObject("mesa") == null ? "-" : "Mesa " + rs.getInt("mesa") %></td>
                    <td><span class="text-muted small"><%= rs.getString("tipo_servicio") %></span></td>
                    <td><span class="badge text-bg-<%= colorEstado(estado) %>"><%= estado %></span></td>
                </tr>
            </tbody>
        </table>
    </div>
    <div>
        <div class="table-responsive">
            <table>
                <thead>
                    <tr>
                        <th>Pedido</th><th>Mesa</th><th>Tipo</th><th>Estado</th>
                        <th class="text-end">Total</th><th>Hora</th><th></th>
                    </tr>
                </thead>
                <tbody>
                    <tr>
                        <td class="fw-semibold"><%= rs.getString("numero_pedido") %></td>
                        <td><%= rs.getObject("mesa") == null ? "-" : "Mesa " + rs.getInt("mesa") %></td>
                        <td><span class="text-muted small"><%= rs.getString("tipo_servicio") %></span></td>
                        <td><span class="badge text-bg-<%= colorEstado(estado) %>"><%= estado %></span></td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>
</div>

Connection con = null; PreparedStatement ps = null; ResultSet rs = null;
int filas = 0;
try {
    con = abrirConexion();
    String sql =
        "SELECT p.id_pedido, p.numero_pedido, p.tipo_servicio, p.estado, "
        + "p.total, DATE_FORMAT(p.fecha_creacion,'%h:%i %p') AS hora, "
        + "m.numero AS mesa "
        + "FROM pedido p LEFT JOIN mesa m ON m.id_mesa = p.id_mesa "
        + "WHERE p.id_mesero = ? AND DATE(p.fecha_creacion) = CURDATE() "
        + "ORDER BY FIELD(p.estado, 'LISTO', 'PENDIENTE', 'CONFIRMADO', "
        + "'EN_PREPARACION', 'SERVIDO', 'PAGADO', 'CANCELADO'), "
        + "p.fecha_creacion DESC";
    ps = con.prepareStatement(sql);
    ps.setInt(1, idUsuarioSesion);
    rs = ps.executeQuery();

    while (rs.next()) {
        filas++;
        String estado = rs.getString("estado");
    }
} catch (Exception e) {
    e.printStackTrace();
}
```

```

        <td class="text-end"><%= pesos(rs.getDouble("total")) %></td>
        <td class="small text-muted"><%= rs.getString("hora") %></td>
        <td class="text-end">
            <a class="btn btn-sm btn-outline-dark"
                href="<%= ctx %>/mesero/pedido.jsp?id=<%= rs.getInt("id_pedido") %>"
                <i class="bi bi-eye"></i> Abrir</a>
        </td>
    </tr>
<%
    }
} catch (SQLException ex) {
%>
    <tr><td colspan="7" class="text-danger">Error: <%= esc(ex.getMessage()) %></td></tr>
<%
    } finally { cerrar(rs, ps, con); }

    if (filas == 0) {
%>
        <tr><td colspan="7" class="text-center text-muted py-4">
            Aun no ha registrado pedidos hoy.</td></tr>
<% } %>
    </tbody>
</table>
</div>
</div>

<%@ include file="/WEB-INF/jspf/pie.jspf" %>

```

La consulta filtra por el mesero de la sesión y por la fecha de hoy. El ORDER BY usa la función FIELD de MySQL para forzar un orden que no es alfabético: primero los pedidos LISTOS (los que hay que recoger ya), después los pendientes y de últimos los cerrados.

Consejo: Observe el patrón que se repite en todas las vistas: abrir conexión, recorrer el ResultSet dentro del HTML con un while, y cerrar en el finally. El contador filas permite mostrar un mensaje amable cuando no hay datos en lugar de una tabla vacía.

Paso 10.2 nuevo.jsp: abrir el pedido

```

web/mesero/nuevo.jsp
<!--
    mesero/nuevo.jsp - Formulario para abrir un pedido nuevo.
    Solo muestra las mesas que estan libres.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"MESERO", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<% String tituloPagina = "Nuevo pedido"; %>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

<h3 class="mb-3"><i class="bi bi-plus-circle"></i> Abrir un pedido nuevo</h3>

<div class="card shadow-sm">
    <div class="card-body">
        <!-- Los datos viajan por POST hacia acciones.jsp con accion=crear -->
        <form method="post" action="<%= ctx %>/mesero/acciones.jsp">
            <input type="hidden" name="accion" value="crear">

            <div class="row g-3">
                <div class="col-md-4">
                    <label class="form-label" for="tipo">Tipo de servicio</label>
                    <select class="form-select" id="tipo" name="tipo_servicio">
                        <option value="MESA" selected>En mesa</option>
                        <option value="LLEVAR">Para llevar</option>
                        <option value="DOMICILIO">Domicilio</option>
                    </select>
                </div>

                <div class="col-md-4">
                    <label class="form-label" for="mesa">Mesa disponible</label>
                    <select class="form-select" id="mesa" name="id_mesa">
                        <option value="">-- Sin mesa (llevar/domicilio) --</option>
                    </select>
                </div>
            </div>

            <div class="form-group">
                <input type="text" name="nombre_cliente" value="Nombre del cliente" />
            </div>

            <div class="form-group">
                <input type="text" name="telefono_cliente" value="Teléfono del cliente" />
            </div>

            <div class="form-group">
                <input type="text" name="direccion_cliente" value="Dirección del cliente" />
            </div>

            <div class="form-group">
                <input type="text" name="comentarios" value="Comentarios" />
            </div>

            <div class="form-group">
                <input type="button" value="Crear pedido" />
            </div>
        </form>
    </div>
</div>

<%
    Connection con = null; Statement st = null; ResultSet rs = null;
    try {
        con = abrirConexion();
        st = con.createStatement();
    }
}

```

```

        // Mesas activas que no tengan un pedido abierto (mesa_activa es NULL
        // cuando el pedido ya se cerro, gracias a la columna generada)
        rs = st.executeQuery(
            "SELECT m.id_mesa, m.numero, m.capacidad, m.ubicacion "
            + "FROM mesa m "
            + "WHERE m.estado <> 'INACTIVA' "
            + " AND m.id_mesa NOT IN (SELECT IFNULL(mesa_activa,0) FROM pedido) "
            + "ORDER BY m.numero");
        while (rs.next()) {
%>
            <option value="<%= rs.getInt("id_mesa") %>">
                Mesa <%= rs.getInt("numero") %>
                (<%= rs.getInt("capacidad") %> puestos - <%= esc(rs.getString("ubicacion")) %>)
            </option>
<%
        }
    } catch (SQLException ex) {
        out.println("<option>Error: " + esc(ex.getMessage()) + "</option>");
    } finally { cerrar(rs, st, con); }
%>
    </select>
</div>

    <div class="col-md-4">
        <label class="form-label" for="comensales">Numero de personas</label>
        <input type="number" class="form-control" id="comensales"
            name="num_comensales" value="1" min="1" max="20">
    </div>

    <div class="col-12">
        <label class="form-label" for="obs">Observaciones generales</label>
        <input type="text" class="form-control" id="obs" name="observaciones"
            maxlength="255" placeholder="Ej: cliente con afan, servir bebidas primero">
    </div>
</div>

<div class="mt-4">
    <button type="submit" class="btn btn-warning fw-bold">
        <i class="bi bi-check2"></i> Crear pedido</button>
    <a href="<%= ctx %>/mesero/pedidos.jsp" class="btn btn-link">Cancelar</a>
</div>
</form>
</div>
</div>
</div>

<%@ include file="/WEB-INF/jspf/pie.jspf" %>

```

La lista de mesas es la parte interesante. No se muestran todas las mesas, solo las que no tienen un pedido abierto, y eso se resuelve con la columna generada mesa_activa del diseño de la base de datos: mientras el pedido está vivo, esa columna repite el id de la mesa; cuando se paga o se cancela, queda en NULL y la mesa vuelve a aparecer en el desplegable.

Paso 10.3 pedido.jsp: la pantalla de trabajo

Es la página más larga del proyecto. Se explica en cuatro bloques.

Bloque 1: cargar el encabezado del pedido

```

web/mesero/pedido.jsp (bloque 1 de 4)
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"MESERO", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%

String tituloPagina = "Detalle del pedido";
int idPedido = aEntero(request.getParameter("id"), 0);
if (idPedido == 0) { response.sendRedirect(ctx + "/mesero/pedidos.jsp"); return; }

// ----- 1) Encabezado del pedido -----
String numeroPedido = "", estado = "", tipoServicio = "", observaciones = "";
String mesa = "-";
double subtotal = 0, impuesto = 0, total = 0;
boolean existe = false;

Connection con = null; PreparedStatement ps = null; ResultSet rs = null;
try {
    con = abrirConexion();
    ps = con.prepareStatement(

```

```

        "SELECT p.numero_pedido, p.estado, p.tipo_servicio, p.observaciones, "
        + "    p.subtotal, p.impuesto, p.total, m.numero AS mesa "
        + "FROM pedido p LEFT JOIN mesa m ON m.id_mesa = p.id_mesa "
        + "WHERE p.id_pedido = ?");
ps.setInt(1, idPedido);
rs = ps.executeQuery();
if (rs.next()) {
    existe = true;
    numeroPedido = rs.getString("numero_pedido");
    estado = rs.getString("estado");
    tipoServicio = rs.getString("tipo_servicio");
    observaciones = rs.getString("observaciones");
    subtotal = rs.getDouble("subtotal");
    impuesto = rs.getDouble("impuesto");
    total = rs.getDouble("total");
    if (rs.getObject("mesa") != null) mesa = "Mesa " + rs.getInt("mesa");
}
cerrar(rs, ps);
} catch (SQLException ex) {
    request.setAttribute("errorBD", ex.getMessage());
}

boolean editable = "PENDIENTE".equals(estado); // solo se edita antes de confirmar
%>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

```

Antes de imprimir nada se consulta el pedido y se guarda su estado. La variable editable decide si la página muestra los controles activos o desactivados: solo un pedido en estado PENDIENTE se puede modificar, porque una vez confirmado la comanda ya está en la cocina.

Bloque 2: encabezado visual

```

web/mesero/pedido.jsp (bloque 2 de 4)
<% if (!existe) { %>
    <div class="alert alert-danger">El pedido solicitado no existe.</div>
<% } else { %>

<div class="d-flex justify-content-between align-items-center mb-3">
    <div>
        <h3 class="mb-0"><%= numeroPedido %>
        <span class="badge text-bg-colorEstado(estado) %> align-middle">
            <%= estado %></span></h3>
        <small class="text-muted"><%= mesa %> &middot; <%= tipoServicio %>
        <%= (observaciones == null || observaciones.isEmpty())
            ? "" : " &middot; " + esc(observaciones) %></small>
    </div>
    <a href="ctx %>/mesero/pedidos.jsp" class="btn btn-outline-secondary btn-sm">
        <i class="bi bi-arrow-left"></i> Volver</a>
    </div>

<% if (request.getParameter("err") != null) { %>
    <div class="alert alert-danger"><%= esc(request.getParameter("err")) %></div>
<% } %>

<div class="row g-4">

```

Bloque 3: la carta de productos

```

web/mesero/pedido.jsp (bloque 3 de 4)
<!-- ===== CARTA DE PRODUCTOS ===== -->
<div class="col-lg-7">
    <div class="card shadow-sm">
        <div class="card-header bg-white fw-bold">
            <i class="bi bi-card-list"></i> Carta
            <% if (!editable) { %>
                <span class="text-danger small ms-2">
                    (el pedido ya fue confirmado: no se puede modificar)</span>
            <% } %>
        </div>
        <div class="card-body p-0">
            <div class="accordion accordion-flush" id="carta">
<%
Statement stCat = null; ResultSet rsCat = null;
try {
    stCat = con.createStatement();
    rsCat = stCat.executeQuery(
        "SELECT id_categoria, nombre FROM categoria "

```

```

+ "WHERE activo = 1 ORDER BY orden_menu");
int i = 0;
while (rsCat.next()) {
    i++;
    int idCat = rsCat.getInt("id_categoria");
%>
    <div class="accordion-item">
        <h2 class="accordion-header">
            <button class="accordion-button <%= i > 1 ? "collapsed" : "" %>"
                type="button" data-bs-toggle="collapse"
                data-bs-target="#cat<%= idCat %>">
                <%= esc(rsCat.getString("nombre")) %>
            </button>
        </h2>
        <div id="cat<%= idCat %>"
            class="accordion-collapse collapse <%= i == 1 ? "show" : "" %>"
            data-bs-parent="#carta">
            <div class="accordion-body p-0">
                <table class="table table-sm mb-0">
%>
                    PreparedStatement psPro = con.prepareStatement(
                        "SELECT id_producto, nombre, precio, tiempo_preparacion "
                        + "FROM producto WHERE id_categoria = ? AND disponible = 1 "
                        + "ORDER BY nombre");
                    psPro.setInt(1, idCat);
                    ResultSet rsPro = psPro.executeQuery();
                    while (rsPro.next()) {
%>
                        <tr class="producto">
                            <td class="align-middle">
                                <b><%= esc(rsPro.getString("nombre")) %></b><br>
                                <small class="text-muted">
                                    <%= pesos(rsPro.getDouble("precio")) %> &middot;
                                    <%= rsPro.getInt("tiempo_preparacion") %> min</small>
                            </td>
                            <td class="align-middle" style="width:60%">
                                <form method="post" action="<%= ctx %>/mesero/acciones.jsp"
                                    class="row g-1">
                                    <input type="hidden" name="accion" value="agregar">
                                    <input type="hidden" name="id_pedido" value="<%= idPedido %>">
                                    <input type="hidden" name="id_producto"
                                        value="<%= rsPro.getInt("id_producto") %>">
                                    <div class="col-3">
                                        <input type="number" name="cantidad" value="1" min="1"
                                            max="50" class="form-control form-control-sm"
                                            <%= editable ? "" : "disabled" %>>
                                    </div>
                                    <div class="col-6">
                                        <input type="text" name="nota" maxlength="150"
                                            class="form-control form-control-sm"
                                            placeholder="Nota para cocina"
                                            <%= editable ? "" : "disabled" %>>
                                    </div>
                                    <div class="col-3 d-grid">
                                        <button class="btn btn-sm btn-warning"
                                            <%= editable ? "" : "disabled" %>>
                                            <i class="bi bi-plus-lg"></i></button>
                                    </div>
                                </form>
                            </td>
                        </tr>
%>
                    }
                    cerrar(rsPro, psPro);
%>
                </table>
            </div>
        </div>
    </div>
%>
}
} catch (SQLException ex) {
    out.println("<div class='alert alert-danger m-3'>" + esc(ex.getMessage()) + "</div>");
} finally { cerrar(rsCat, stCat); }
%>
    </div>
</div>
</div>
</div>

```

La carta se arma con dos consultas anidadas: una recorre las categorías y, dentro de cada una, otra trae sus productos disponibles. Cada producto tiene su propio formulario con la cantidad y la nota para cocina, de modo que agregar un ítem es un POST independiente.

Elemento	Explicación
<code>accordion / accordion-item</code>	Componente de Bootstrap que agrupa las categorías en secciones plegables.
<code>data-bs-target="#cat<%= idCat %>"</code>	El identificador se genera con el id de la categoría para que cada panel sea único.
<code><%= editable ? "" : "disabled" %></code>	Escritura condicional de un atributo HTML: los campos se bloquean cuando el pedido ya no admite cambios.
Formulario por producto	Evita JavaScript: cada botón envía su propio POST a acciones.jsp con accion=agregar.

Bloque 4: detalle, totales y botones de estado

```

web/mesero/pedido.jsp (bloque 4 de 4)
<!-- ===== DETALLE DEL PEDIDO ===== -->
<div class="col-lg-5">
    <div class="card shadow-sm">
        <div class="card-header bg-white fw-bold">
            <i class="bi bi-basket"></i> Productos del pedido</div>
        <div class="table-responsive">
            <table class="table table-sm align-middle mb-0">
                <thead class="table-light">
                    <tr><th>Producto</th><th class="text-center">Cant.</th>
                    <th class="text-end">Subtotal</th><th></th></tr>
                </thead>
                <tbody>
                    <tr>
                        <td>
                            <%= esc(rs.getString("nombre")) %>
                            <% if (rs.getString("nota") != null && !rs.getString("nota").isEmpty()) { %>
                                <br><small class="text-muted fst-italic">
                                    <i class="bi bi-chat-left-text"></i>
                                    <%= esc(rs.getString("nota")) %></small>
                            <% } %>
                            <br><span class="badge text-bg-<%= colorEstado(rs.getString("estado_item")) %>">
                                <%= rs.getString("estado_item") %></span>
                        </td>
                        <td class="text-center"><%= rs.getInt("cantidad") %></td>
                        <td class="text-end"><%= pesos(rs.getDouble("subtotal")) %></td>
                        <td class="text-end">
                            <% if (editable) { %>
                                <form method="post" action="<%= ctx %>/mesero/acciones.jsp"
                                    onsubmit="return confirm('Quitar este producto del pedido?')">
                                    <input type="hidden" name="accion" value="quitar">
                                    <input type="hidden" name="id_pedido" value="<%= idPedido %>">
                                    <input type="hidden" name="id_detalle"
                                        value="<%= rs.getInt("id_detalle") %>">
                                    <button class="btn btn-sm btn-outline-danger">
                                        <i class="bi bi-trash"></i></button>
                                </form>
                            <% } %>
                        </td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>
</div>
<%
int lineas = 0;
try {
    ps = con.prepareStatement(
        "SELECT d.id_detalle, d.cantidad, d.precio_unitario, d.subtotal, "
        + "d.nota, d.estado_item, pr.nombre "
        + "FROM detalle_pedido d JOIN producto pr ON pr.id_producto = d.id_producto "
        + "WHERE d.id_pedido = ? ORDER BY d.id_detalle");
    ps.setInt(1, idPedido);
    rs = ps.executeQuery();
    while (rs.next()) {
        lineas++;
        <tr>
            <td>
                <%= esc(rs.getString("nombre")) %>
                <% if (rs.getString("nota") != null && !rs.getString("nota").isEmpty()) { %>
                    <br><small class="text-muted fst-italic">
                        <i class="bi bi-chat-left-text"></i>
                        <%= esc(rs.getString("nota")) %></small>
                <% } %>
                <br><span class="badge text-bg-<%= colorEstado(rs.getString("estado_item")) %>">
                    <%= rs.getString("estado_item") %></span>
            </td>
            <td class="text-center"><%= rs.getInt("cantidad") %></td>
            <td class="text-end"><%= pesos(rs.getDouble("subtotal")) %></td>
            <td class="text-end">
                <% if (editable) { %>
                    <form method="post" action="<%= ctx %>/mesero/acciones.jsp"
                        onsubmit="return confirm('Quitar este producto del pedido?')">
                        <input type="hidden" name="accion" value="quitar">
                        <input type="hidden" name="id_pedido" value="<%= idPedido %>">
                        <input type="hidden" name="id_detalle"
                            value="<%= rs.getInt("id_detalle") %>">
                        <button class="btn btn-sm btn-outline-danger">
                            <i class="bi bi-trash"></i></button>
                    </form>
                <% } %>
            </td>
        </tr>
    }
} catch (SQLException ex) {
    out.println("<tr><td colspan='4' class='text-danger'"
        + esc(ex.getMessage()) + "</td></tr>");
} finally { cerrar(rs, ps, con); }

if (lineas == 0) {
    <tr><td colspan="4" class="text-center text-muted py-4">

```

```

        El pedido esta vacio. Agregue productos desde la carta.</td></tr>
<% } %>
</tbody>
<tfoot class="table-light">
<tr><td colspan="2" class="text-end">Subtotal</td>
<td class="text-end"><%= pesos(subtotal) %></td></td></tr>
<tr><td colspan="2" class="text-end">Impuesto (8%)</td>
<td class="text-end"><%= pesos(impuesto) %></td></td></tr>
<tr class="fw-bold fs-5"><td colspan="2" class="text-end">Total</td>
<td class="text-end"><%= pesos(total) %></td></td></tr>
</tfoot>
</table>
</div>

<!-- ===== Botones segun el estado del pedido ===== -->
<div class="card-body border-top">
<% if ("PENDIENTE".equals(estado)) { %>
<form method="post" action="<%= ctx %>/mesero/acciones.jsp" class="d-grid gap-2">
<input type="hidden" name="accion" value="confirmar">
<input type="hidden" name="id_pedido" value="<%= idPedido %>">
<button class="btn btn-success btn-lg" <%= lineas == 0 ? "disabled" : "" %>>
<i class="bi bi-send"></i> Confirmar y enviar a cocina</button>
</form>
<form method="post" action="<%= ctx %>/mesero/acciones.jsp" class="d-grid mt-2"
onsubmit="return confirm('Cancelar definitivamente el pedido?')">
<input type="hidden" name="accion" value="cancelar">
<input type="hidden" name="id_pedido" value="<%= idPedido %>">
<input type="hidden" name="motivo" value="Cancelado por el mesero">
<button class="btn btn-outline-danger btn-sm">Cancelar pedido</button>
</form>

<% } else if ("CONFIRMADO".equals(estado)) { %>
<div class="alert alert-warning mb-0">
<i class="bi bi-hourglass-split"></i>
Enviado a cocina. Esperando que lo tomen.</div>

<% } else if ("EN_PREPARACION".equals(estado)) { %>
<div class="alert alert-info mb-0">
<i class="bi bi-fire"></i> La cocina lo esta preparando.</div>

<% } else if ("LISTO".equals(estado)) { %>
<form method="post" action="<%= ctx %>/mesero/acciones.jsp" class="d-grid">
<input type="hidden" name="accion" value="servir">
<input type="hidden" name="id_pedido" value="<%= idPedido %>">
<button class="btn btn-primary btn-lg">
<i class="bi bi-check2-all"></i> Marcar como servido</button>
</form>

<% } else if ("SERVIDO".equals(estado)) { %>
<a href="<%= ctx %>/caja/facturar.jsp?id=<%= idPedido %>"
class="btn btn-success btn-lg w-100">
<i class="bi bi-cash-coin"></i> Ir a facturar</a>

<% } else { %>
<div class="alert alert-secondary mb-0">
Pedido <%= estado.toLowerCase() %>. No admite mas cambios.</div>
<% } %>
</div>
</div>
</div>
<% } %>

<%@ include file="/WEB-INF/jspf/pie.jspf" %>

```

El pie de la tabla muestra subtotal, impuesto y total tal como están guardados en la tabla pedido; no se calculan en la página. Los mantiene actualizados el procedimiento `sp_recalcular_total` cada vez que cambia el detalle.

La botonera cambia según el estado: en PENDIENTE aparecen confirmar y cancelar; en CONFIRMADO y EN_PREPARACION solo un aviso informativo; en LISTO el botón de servir; y en SERVIDO el enlace a la caja. Esto refleja exactamente la tabla de transiciones del diseño de la base de datos.

Paso 10.4 acciones.jsp: el controlador del mesero

Aquí ocurre todo lo que modifica la base de datos. Es un buen ejemplo de controlador sencillo: recibe el parámetro accion, ejecuta la operación dentro de una transacción y termina siempre con una redirección.

Estructura general

web/mesero/acciones.jsp (parte 1: preparación)

```
<!--
mesero/acciones.jsp - Controlador del modulo mesero.
Recibe por POST el parametro "accion" y ejecuta la operacion sobre la base
de datos dentro de una transaccion. No produce HTML: siempre redirige.

Acciones: crear | agregar | quitar | confirmar | servir | cancelar
--%>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"MESERO", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%

request.setCharacterEncoding("UTF-8");
String accion = request.getParameter("accion");
int idPedido = aEntero(request.getParameter("id_pedido"), 0);

Connection con = null;
PreparedStatement ps = null;
ResultSet rs = null;
String destino;

try {
    con = abrirConexion();
    con.setAutoCommit(false);          // ---- inicia la transaccion ----
```

Fíjese en setAutoCommit(false): a partir de esa línea nada queda grabado hasta que se llame a commit(). Si en el camino aparece una excepción, el bloque catch llama a deshacer(con), que ejecuta el rollback, y la base de datos vuelve a como estaba.

Acción 1: crear el pedido

web/mesero/acciones.jsp (parte 2: crear)

```
// ===== 1) CREAR: abre un pedido nuevo y ocupa la mesa =====
if ("crear".equals(accion)) {
    String tipo = request.getParameter("tipo_servicio");
    String mesaTxt = request.getParameter("id_mesa");
    int comensales = aEntero(request.getParameter("num_comensales"), 1);
    String obs = request.getParameter("observaciones");

    // Consecutivo del día: PED-AAAAMDD-001
    String numero;
    ps = con.prepareStatement(
        "SELECT CONCAT('PED-', DATE_FORMAT(CURDATE(), '%Y%m%d'), '-', "
        + "LPAD(COUNT(*) + 1, 3, '0')) AS consecutivo "
        + "FROM pedido WHERE DATE(fecha_creacion) = CURDATE()");
    rs = ps.executeQuery();
    rs.next();
    numero = rs.getString("consecutivo");
    cerrar(rs, ps);

    ps = con.prepareStatement(
        "INSERT INTO pedido (numero_pedido, tipo_servicio, id_mesa, "
        + "id_mesero, num_comensales, observaciones) "
        + "VALUES (?, ?, ?, ?, ?, ?)",
        Statement.RETURN_GENERATED_KEYS);
    ps.setString(1, numero);
    ps.setString(2, tipo);
    if (mesaTxt == null || mesaTxt.trim().isEmpty()) ps.setNull(3, Types.INTEGER);
    else ps.setInt(3, aEntero(mesaTxt, 0));
    ps.setInt(4, idUsuarioSesion);
    ps.setInt(5, comensales);
    ps.setString(6, obs);
    ps.executeUpdate();

    rs = ps.getGeneratedKeys();
    rs.next();
```

```

int nuevoId = rs.getInt(1);
cerrar(rs, ps);

// La mesa queda ocupada
if (mesaTxt != null && !mesaTxt.trim().isEmpty()) {
    ps = con.prepareStatement(
        "UPDATE mesa SET estado = 'OCUPADA' WHERE id_mesa = ?");
    ps.setInt(1, aEntero(mesaTxt, 0));
    ps.executeUpdate();
    cerrar(ps);
}

registrarSeguimiento(con, nuevoId, null, "PENDIENTE",
    idUsuarioSesion, "Pedido creado");
con.commit();
destino = ctx + "/mesero/pedido.jsp?id=" + nuevoId;

```

Instrucción	Explicación
<code>CONCAT('PED-', DATE_FORMAT(...), LPAD(COUNT(*)+1,3,'0'))</code>	Arma el consecutivo del día en la propia base de datos: PED-20260901-001.
<code>Statement.RETURN_GENERATED_KEYS</code>	Pide a JDBC que devuelva el id autogenerado del pedido recién insertado.
<code>ps.setNull(3, Types.INTEGER)</code>	Un pedido para llevar no tiene mesa: hay que enviar NULL explícitamente, no una cadena vacía.
<code>UPDATE mesa SET estado='OCUPADA'</code>	Refleja en el salón que la mesa está tomada.
<code>registrarSeguimiento(...)</code>	Primera línea de la bitácora del pedido.

Acciones 2 y 3: agregar y quitar productos

web/mesero/acciones.jsp (parte 3: agregar y quitar)

```

// ===== 2) AGREGAR un producto al detalle =====
} else if ("agregar".equals(accion)) {
    int idProducto = aEntero(request.getParameter("id_producto"), 0);
    int cantidad = aEntero(request.getParameter("cantidad"), 1);
    String nota = request.getParameter("nota");

    // El UNIQUE (id_pedido, id_producto) impide líneas repetidas:
    // si el producto ya estaba, ON DUPLICATE KEY suma la cantidad.
    ps = con.prepareStatement(
        "INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad, "
        + "                                precio_unitario, nota) "
        + "VALUES (?, ?, ?, (SELECT precio FROM producto WHERE id_producto = ?), ?) "
        + "ON DUPLICATE KEY UPDATE cantidad = cantidad + VALUES(cantidad)");
    ps.setInt(1, idPedido);
    ps.setInt(2, idProducto);
    ps.setInt(3, cantidad);
    ps.setInt(4, idProducto);
    ps.setString(5, (nota == null || nota.trim().isEmpty()) ? null : nota.trim());
    ps.executeUpdate();
    cerrar(ps);

    recalcularTotal(con, idPedido);
    con.commit();
    destino = ctx + "/mesero/pedido.jsp?id=" + idPedido;

// ===== 3) QUITAR una línea del detalle =====
} else if ("quitar".equals(accion)) {
    int idDetalle = aEntero(request.getParameter("id_detalle"), 0);
    ps = con.prepareStatement(
        "DELETE FROM detalle_pedido WHERE id_detalle = ? AND id_pedido = ?");
    ps.setInt(1, idDetalle);
    ps.setInt(2, idPedido);
    ps.executeUpdate();
    cerrar(ps);

    recalcularTotal(con, idPedido);
    con.commit();
    destino = ctx + "/mesero/pedido.jsp?id=" + idPedido;
}

```

Importante: La cláusula ON DUPLICATE KEY UPDATE es posible gracias a la restricción UNIQUE (id_pedido, id_producto). Si el mesero agrega dos veces la misma hamburguesa, MySQL detecta el choque con esa restricción y en lugar de fallar suma la

cantidad en la línea que ya existía. Es el ejemplo más claro de cómo un buen diseño de base de datos simplifica el código de la aplicación.

El precio no se envía desde el formulario: se toma con una subconsulta directamente de la tabla producto. Nunca se debe confiar en un precio que viaja por el navegador, porque cualquiera podría modificarlo con las herramientas de desarrollo.

Acciones 4 y 5: confirmar y servir

```
web/mesero/acciones.jsp (parte 4: confirmar y servir)
// ===== 4) CONFIRMAR: el pedido pasa a la cocina =====
} else if ("confirmar".equals(accion)) {
    ps = con.prepareStatement(
        "UPDATE pedido SET estado = 'CONFIRMADO', fecha_confirmacion = NOW() "
        + "WHERE id_pedido = ? AND estado = 'PENDIENTE'");
    ps.setInt(1, idPedido);
    int filas = ps.executeUpdate();
    cerrar(ps);

    if (filas == 0) {
        deshacer(con);
        destino = ctx + "/mesero/pedido.jsp?id=" + idPedido
            + "&err=" + java.net.URLEncoder.encode(
                "El pedido ya no esta pendiente.", "UTF-8");
    } else {
        registrarSeguimiento(con, idPedido, "PENDIENTE", "CONFIRMADO",
            idUsuarioSesion, "Enviado a cocina");
        con.commit();
        destino = ctx + "/mesero/pedidos.jsp?msg="
            + java.net.URLEncoder.encode(
                "Pedido enviado a cocina.", "UTF-8");
    }
}

// ===== 5) SERVIR: el mesero entrega el pedido en la mesa =====
} else if ("servir".equals(accion)) {
    ps = con.prepareStatement(
        "UPDATE pedido SET estado = 'SERVIDO', fecha_servido = NOW() "
        + "WHERE id_pedido = ? AND estado = 'LISTO'");
    ps.setInt(1, idPedido);
    int filas = ps.executeUpdate();
    cerrar(ps);

    if (filas == 0) {
        deshacer(con);
        destino = ctx + "/mesero/pedido.jsp?id=" + idPedido
            + "&err=" + java.net.URLEncoder.encode(
                "El pedido todavia no esta listo.", "UTF-8");
    } else {
        ps = con.prepareStatement(
            "UPDATE detalle_pedido SET estado_item = 'SERVIDO' "
            + "WHERE id_pedido = ? AND estado_item = 'LISTO'");
        ps.setInt(1, idPedido);
        ps.executeUpdate();
        cerrar(ps);

        registrarSeguimiento(con, idPedido, "LISTO", "SERVIDO",
            idUsuarioSesion, "Entregado en la mesa");
        con.commit();
        destino = ctx + "/mesero/pedido.jsp?id=" + idPedido;
    }
}
```

Las dos operaciones usan el mismo truco de seguridad: el UPDATE incluye la condición del estado esperado (AND estado = 'PENDIENTE'). Si otro usuario ya cambió el pedido, executeUpdate devuelve 0 filas, la transacción se deshace y se avisa al mesero. Así se evita que dos personas trabajando al tiempo dejen el pedido en un estado incoherente.

Acción 6: cancelar, y el cierre del controlador

```
web/mesero/acciones.jsp (parte 5: cancelar y cierre)
// ===== 6) CANCELAR: solo antes de que cocina lo tome =====
} else if ("cancelar".equals(accion)) {
    String motivo = request.getParameter("motivo");

    ps = con.prepareStatement(
        "SELECT estado, id_mesa FROM pedido WHERE id_pedido = ?");
    ps.setInt(1, idPedido);
    rs = ps.executeQuery();
```

```

String estadoActual = null; Integer idMesa = null;
if (rs.next()) {
    estadoActual = rs.getString("estado");
    idMesa = (Integer) rs.getObject("id_mesa");
}
cerrar(rs, ps);

if (!"PENDIENTE".equals(estadoActual) && !"CONFIRMADO".equals(estadoActual)) {
    deshacer(con);
    destino = ctx + "/mesero/pedido.jsp?id=" + idPedido
        + "&err=" + java.net.URLEncoder.encode(
            "Ya no se puede cancelar: la cocina lo tomo.", "UTF-8");
} else {
    ps = con.prepareStatement(
        "UPDATE pedido SET estado = 'CANCELADO', fecha_cierre = NOW() "
        + "WHERE id_pedido = ?");
    ps.setInt(1, idPedido);
    ps.executeUpdate();
    cerrar(ps);

    if (idMesa != null) {
        ps = con.prepareStatement(
            "UPDATE mesa SET estado = 'LIBRE' WHERE id_mesa = ?");
        ps.setInt(1, idMesa);
        ps.executeUpdate();
        cerrar(ps);
    }
    registrarSeguimiento(con, idPedido, estadoActual, "CANCELADO",
        idUsuarioSesion, motivo);
    con.commit();
    destino = ctx + "/mesero/pedidos.jsp?msg="
        + java.net.URLEncoder.encode("Pedido cancelado.", "UTF-8");
}

} else {
    destino = ctx + "/mesero/pedidos.jsp";
}

} catch (SQLException ex) {
    deshacer(con); // ---- rollback ----
    destino = ctx + "/mesero/pedidos.jsp?err="
        + java.net.URLEncoder.encode(mensajeError(ex), "UTF-8");
} finally {
    cerrar(rs, ps, con);
}
response.sendRedirect(destino);
%>

```

Cancelar solo se permite mientras la cocina no haya tomado el pedido. Además de cambiar el estado, libera la mesa y guarda el motivo en la bitácora.

Métodos auxiliares del controlador

web/mesero/acciones.jsp (parte 6: métodos)

```

<%!
/* ----- Metodos auxiliares del controlador ----- */

/** Inserta el cambio de estado en la bitacora seguimiento_pedido. */
void registrarSeguimiento(Connection con, int idPedido, String anterior,
    String nuevo, int idUsuario, String comentario)
    throws SQLException {
    PreparedStatement ps = con.prepareStatement(
        "INSERT INTO seguimiento_pedido (id_pedido, id_usuario, "
        + "    estado_anterior, estado_nuevo, comentario) "
        + "VALUES (?, ?, ?, ?, ?)");
    ps.setInt(1, idPedido);
    ps.setInt(2, idUsuario);
    ps.setString(3, anterior);
    ps.setString(4, nuevo);
    ps.setString(5, comentario);
    ps.executeUpdate();
    ps.close();
}

/** Llama al procedimiento almacenado que recalcula subtotal, impuesto y total. */
void recalcularTotal(Connection con, int idPedido) throws SQLException {
    CallableStatement cs = con.prepareCall("{ CALL sp_recalcular_total(?) }");
    cs.setInt(1, idPedido);
    cs.execute();
    cs.close();
}

```

```

}

/** Traduce los errores de restriccion a un mensaje entendible. */
String mensajeError(SQLException ex) {
    String m = ex.getMessage() == null ? "" : ex.getMessage();
    if (m.contains("uq_pedido_mesa_activa"))
        return "Esa mesa ya tiene un pedido abierto.";
    if (m.contains("uq_seguimiento_estado"))
        return "Ese cambio de estado ya fue registrado.";
    if (m.contains("uq_detalle_producto"))
        return "El producto ya esta en el pedido.";
    if (m.contains("sp_recalcular_total"))
        return "Falta crear el procedimiento sp_recalcular_total en la base de datos.";
    return "Error de base de datos: " + m;
}
%>

```

Método	Explicación
registrarSeguimiento(...)	Inserta el cambio de estado en seguimiento_pedido con el usuario y la hora. Al ser un método, la bitácora nunca se olvida.
recalcularTotal(...)	Llama con CallableStatement al procedimiento almacenado sp_recalcular_total, que recalcula subtotal, impuesto y total del pedido.
mensajeError(SQLException)	Traduce el nombre técnico de la restricción violada a una frase que el mesero entiende. Compare el mensaje del motor con el que ve el usuario.

Paso 11. Módulo de cocina

La cocina no necesita una interfaz complicada: una pantalla con las comandas en tarjetas grandes, ordenadas por antigüedad, y dos botones. Como la pantalla suele estar colgada en la pared, la página se recarga sola.

Paso 11.1 tablero.jsp

```
web/cocina/tablero.jsp
<!--
cocina/tablero.jsp - Pantalla de la cocina.
Muestra como tarjetas las comandas CONFIRMADAS y EN_PREPARACION.
Se refresca sola cada 20 segundos para que no haya que recargar a mano.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"COCINA", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<% String tituloPagina = "Tablero de cocina"; %>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

<!-- Recarga automatica cada 20 segundos -->
<meta http-equiv="refresh" content="20">

<div class="d-flex justify-content-between align-items-center mb-3">
  <h3 class="mb-0"><i class="bi bi-fire text-info"></i> Comandas en cocina</h3>
  <span class="text-muted small">
    <i class="bi bi-arrow-clockwise"></i> Se actualiza cada 20 segundos</span>
</div>

<% if (request.getParameter("err") != null) { %>
  <div class="alert alert-danger"><%= esc(request.getParameter("err")) %></div>
<% } %>

<div class="row g-3">
  <%
    Connection con = null; PreparedStatement ps = null; ResultSet rs = null;
    int comandas = 0;
    try {
      con = abrirConexion();
      ps = con.prepareStatement(
        "SELECT p.id_pedido, p.numero_pedido, p.estado, p.observaciones, "
        + "      p.tipo_servicio, m.numero AS mesa, "
        + "      CONCAT(u.nombres, ' ', u.apellidos) AS mesero, "
        + "      TIMESTAMPDIFF(MINUTE, p.fecha_confirmacion, NOW()) AS minutos "
        + "FROM pedido p "
        + "      JOIN usuario u ON u.id_usuario = p.id_mesero "
        + "      LEFT JOIN mesa m ON m.id_mesa = p.id_mesa "
        + "WHERE p.estado IN ('CONFIRMADO', 'EN_PREPARACION') "
        + "ORDER BY p.fecha_confirmacion ASC");
      rs = ps.executeQuery();

      while (rs.next()) {
        comandas++;
        int idPedido = rs.getInt("id_pedido");
        String estado = rs.getString("estado");
        int minutos = rs.getInt("minutos");
        String clase = minutos >= 15 ? "tarde" : ("CONFIRMADO".equals(estado)
          ? "espera" : "");

        <div class="col-md-6 col-xl-4">
          <div class="card shadow-sm h-100 comanda <%= clase %>">
            <div class="card-header bg-white d-flex justify-content-between">
              <b><%= rs.getString("numero_pedido") %></b>
              <span class="badge text-bg-<%= colorEstado(estado) %>"><%= estado %></span>
            </div>
            <div class="card-body">
              <p class="mb-2 small text-muted">
                <i class="bi bi-geo-alt"></i>
                <%= rs.getObject("mesa") == null
                  ? rs.getString("tipo_servicio") : "Mesa " + rs.getInt("mesa") %>
                &middot; <i class="bi bi-person"></i> <%= esc(rs.getString("mesero")) %>
                &middot; <i class="bi bi-clock"></i>
                <b class="<%= minutos >= 15 ? "text-danger" : "" %>"><%= minutos %> min</b>
              </p>

              <ul class="list-group list-group-flush mb-2">
                PreparedStatement psDet = con.prepareStatement(
```

```

        "SELECT d.cantidad, d.nota, pr.nombre "
        + "FROM detalle_pedido d "
        + "    JOIN producto pr ON pr.id_producto = d.id_producto "
        + "WHERE d.id_pedido = ? AND d.estado_item <> 'ANULADO' "
        + "ORDER BY d.id_detalle");
psDet.setInt(1, idPedido);
ResultSet rsDet = psDet.executeQuery();
while (rsDet.next()) {
%>
    <li class="list-group-item px-0 py-1">
        <span class="badge text-bg-dark"><%= rsDet.getInt("cantidad") %></span>
        <%= esc(rsDet.getString("nombre")) %>
        <% if (rsDet.getString("nota") != null
            && !rsDet.getString("nota").isEmpty()) { %>
            <br><small class="text-danger fst-italic ms-4">
                <i class="bi bi-exclamation-circle"></i>
                <%= esc(rsDet.getString("nota")) %></small>
            <% } %>
        </li>
<%
    }
    cerrar(rsDet, psDet);

    String obs = rs.getString("observaciones");
    if (obs != null && !obs.isEmpty()) {
%>
        </ul>
        <div class="alert alert-warning py-1 px-2 small mb-2">
            <i class="bi bi-info-circle"></i> <%= esc(obs) %></div>
        <%
    } else { %>
        </ul>
    } %>

    <!-- Botones de la comanda -->
    <% if ("CONFIRMADO".equals(estado)) { %>
        <form method="post" action="<%= ctx %>/cocina/accion.jsp" class="d-grid">
            <input type="hidden" name="accion" value="tomar">
            <input type="hidden" name="id_pedido" value="<%= idPedido %>">
            <button class="btn btn-info fw-bold">
                <i class="bi bi-play-fill"></i> Iniciar preparacion</button>
        </form>
    } else { %>
        <form method="post" action="<%= ctx %>/cocina/accion.jsp" class="d-grid">
            <input type="hidden" name="accion" value="listo">
            <input type="hidden" name="id_pedido" value="<%= idPedido %>">
            <button class="btn btn-success fw-bold">
                <i class="bi bi-bell"></i> Marcar como listo</button>
        </form>
    } %>
</div>
</div>
</div>
<%
}
} catch (SQLException ex) {
    out.println("<div class='alert alert-danger'" + esc(ex.getMessage()) + "</div>");
} finally { cerrar(rs, ps, con); }

    if (comandas == 0) {
%>
    <div class="col-12">
        <div class="alert alert-secondary text-center py-5">
            <i class="bi bi-emoji-smile fs-1"></i>
            <p class="mb-0 mt-2">No hay comandas pendientes. Cocina al dia.</p>
        </div>
    </div>
    <% } %>
</div>

<%@ include file="/WEB-INF/jspf/pie.jspf" %>

```

Detalle	Explicación
<meta http-equiv="refresh" content="20">	Recarga la página cada 20 segundos, así aparecen solas las comandas nuevas sin necesidad de JavaScript ni AJAX.
TIMESTAMPDIFF(MINUTE, fecha_confirmacion, NOW())	Calcula en la propia consulta cuántos minutos lleva esperando la comanda.
clase tarde / espera	Si la comanda pasa de 15 minutos, el borde de la tarjeta se pinta de rojo con la regla de estilos.css.

Consulta anidada del detalle	Por cada pedido se abre un segundo PreparedStatement que trae sus productos y sus notas.
Nota en rojo	La nota del cliente ("sin cebolla") se resalta porque es la información que más se pasa por alto en cocina.
Botón según el estado	CONFIRMADO muestra "Iniciar preparación"; EN_PREPARACION muestra "Marcar como listo".

Paso 11.2 accion.jsp

```
web/cocina/accion.jsp
<!--
cocina/accion.jsp - Controlador del modulo de cocina.
Acciones: tomar (CONFIRMADO -> EN_PREPARACION) y listo (-> LISTO).
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"COCINA", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
String accion = request.getParameter("accion");
int idPedido = aEntero(request.getParameter("id_pedido"), 0);
String destino = ctx + "/cocina/tablero.jsp";

Connection con = null; PreparedStatement ps = null;
try {
    con = abrirConexion();
    con.setAutoCommit(false);

    if ("tomar".equals(accion)) {
        // Solo avanza si sigue CONFIRMADO: evita que dos cocineros
        // tomen la misma comanda al mismo tiempo.
        ps = con.prepareStatement(
            "UPDATE pedido SET estado = 'EN_PREPARACION', id_cocinero = ? "
            + "WHERE id_pedido = ? AND estado = 'CONFIRMADO'");
        ps.setInt(1, idUsuarioSesion);
        ps.setInt(2, idPedido);
        int filas = ps.executeUpdate();
        cerrar(ps);

        if (filas == 0) {
            deshacer(con);
            destino += "?err=" + java.net.URLEncoder.encode(
                "Otro cocinero ya tomo ese pedido.", "UTF-8");
        } else {
            ps = con.prepareStatement(
                "UPDATE detalle_pedido SET estado_item = 'EN_PREPARACION' "
                + "WHERE id_pedido = ? AND estado_item = 'PENDIENTE'");
            ps.setInt(1, idPedido);
            ps.executeUpdate();
            cerrar(ps);

            ps = con.prepareStatement(
                "INSERT INTO seguimiento_pedido (id_pedido, id_usuario, "
                + "estado_anterior, estado_nuevo, comentario) "
                + "VALUES (?, ?, 'CONFIRMADO', 'EN_PREPARACION', 'Cocina inicio')");
            ps.setInt(1, idPedido);
            ps.setInt(2, idUsuarioSesion);
            ps.executeUpdate();
            cerrar(ps);
            con.commit();
        }
    }

    } else if ("listo".equals(accion)) {
        ps = con.prepareStatement(
            "UPDATE pedido SET estado = 'LISTO', fecha_listo = NOW() "
            + "WHERE id_pedido = ? AND estado = 'EN_PREPARACION'");
        ps.setInt(1, idPedido);
        int filas = ps.executeUpdate();
        cerrar(ps);

        if (filas == 0) {
            deshacer(con);
            destino += "?err=" + java.net.URLEncoder.encode(
                "El pedido no esta en preparacion.", "UTF-8");
        } else {
            ps = con.prepareStatement(
                "UPDATE detalle_pedido SET estado_item = 'LISTO' "
```

```

        + "WHERE id_pedido = ? AND estado_item = 'EN_PREPARACION'");
ps.setInt(1, idPedido);
ps.executeUpdate();
cerrar(ps);

ps = con.prepareStatement(
    "INSERT INTO seguimiento_pedido (id_pedido, id_usuario, "
    + "        estado_anterior, estado_nuevo, comentario) "
    + "VALUES (?, ?, 'EN_PREPARACION', 'LISTO', 'Preparacion terminada')");
ps.setInt(1, idPedido);
ps.setInt(2, idUsuarioSesion);
ps.executeUpdate();
cerrar(ps);
con.commit();
    }
}
} catch (SQLException ex) {
    deshacer(con);
    destino += "?err=" + java.net.URLEncoder.encode(ex.getMessage(), "UTF-8");
} finally {
    cerrar(ps, con);
}
response.sendRedirect(destino);
%>

```

La acción tomar guarda el id del cocinero en el pedido y cambia el estado de las líneas del detalle. Igual que en el módulo del mesero, el UPDATE lleva la condición del estado esperado: si dos cocineros pulsán el botón al mismo tiempo, solo uno de los dos verá su cambio aplicado y el otro recibirá el aviso "Otro cocinero ya tomó ese pedido".

Consejo: Cambie el intervalo del meta refresh a 10 segundos si va a hacer la demostración en clase: la reacción del tablero se aprecia mucho mejor.

Paso 12. Módulo de caja

El cierre del ciclo: se cobra el pedido servido, se genera la factura, se libera la mesa y se imprime el comprobante.

Paso 12.1 cobrar.jsp

```
web/caja/cobrar.jsp
<!--
  caja/cobrar.jsp - Lista los pedidos servidos que faltan por facturar.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"CAJERO", "MESERO", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<% String tituloPagina = "Caja"; %>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

<h3 class="mb-3"><i class="bi bi-cash-coin text-success"></i> Pedidos por cobrar</h3>

<% if (request.getParameter("msg") != null) { %>
  <div class="alert alert-success"><%= esc(request.getParameter("msg")) %></div>
<% } %>

<div class="card shadow-sm">
  <div class="table-responsive">
    <table class="table table-hover align-middle mb-0">
      <thead class="table-dark">
        <tr><th>Pedido</th><th>Mesa</th><th>Mesero</th>
          <th class="text-end">Total</th><th></th></tr>
      </thead>
      <tbody>
<%
  Connection con = null; Statement st = null; ResultSet rs = null;
  int filas = 0;
  try {
    con = abrirConexion();
    st = con.createStatement();
    rs = st.executeQuery(
      "SELECT p.id_pedido, p.numero_pedido, p.total, m.numero AS mesa, "
      + "      CONCAT(u.nombres, ' ', u.apellidos) AS mesero "
      + "FROM pedido p "
      + "      JOIN usuario u ON u.id_usuario = p.id_mesero "
      + "      LEFT JOIN mesa m ON m.id_mesa = p.id_mesa "
      + "WHERE p.estado = 'SERVIDO' "
      + "ORDER BY p.fecha_servido");
    while (rs.next()) {
      filas++;
<%
      <tr>
        <td class="fw-semibold"><%= rs.getString("numero_pedido") %></td>
        <td><%= rs.getObject("mesa") == null ? "-" : "Mesa " + rs.getInt("mesa") %></td>
        <td class="small"><%= esc(rs.getString("mesero")) %></td>
        <td class="text-end fw-bold"><%= pesos(rs.getDouble("total")) %></td>
        <td class="text-end">
          <a class="btn btn-sm btn-success"
            href="<%= ctx %>/caja/facturar.jsp?id=<%= rs.getInt("id_pedido") %>"
            <i class="bi bi-receipt-cutoff"></i> Facturar</a>
        </td>
      </tr>
<%
    }
  } catch (SQLException ex) {
    out.println("<tr><td colspan='5' class='text-danger'"
      + esc(ex.getMessage()) + "</td></tr>");
  } finally { cerrar(rs, st, con); }

  if (filas == 0) {
<%
    <tr><td colspan="5" class="text-center text-muted py-4">
      No hay pedidos servidos pendientes de pago.</td></tr>
  <% } %>
  </tbody>
</table>
</div>
</div>

<%@ include file="/WEB-INF/jspf/pie.jspf" %>
```

Paso 12.2 facturar.jsp

Esta página hace dos cosas en un solo archivo: si se abre con GET muestra el formulario de pago; si llega un POST, registra la factura y muestra el comprobante. La primera parte es el procesamiento.

web/caja/facturar.jsp (parte 1: procesamiento del pago)

```
<!--
caja/facturar.jsp - Muestra el formulario de pago, procesa la factura
dentro de una transaccion y presenta el comprobante imprimible.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"CAJERO", "MESERO", "ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%

request.setCharacterEncoding("UTF-8");
String tituloPagina = "Facturar";
int idPedido = aEntero(request.getParameter("id"), 0);
String error = null, numeroFactura = null;
double cambio = 0, recibido = 0;
boolean facturado = false;

String numeroPedido = "", estado = "", mesa = "-";
double total = 0;

Connection con = null; PreparedStatement ps = null; ResultSet rs = null;
try {
    con = abrirConexion();

    /* ----- A) Si llega por POST: registrar el pago ----- */
    if ("POST".equalsIgnoreCase(request.getMethod())) {
        String metodo = request.getParameter("metodo_pago");
        recibido = aDoble(request.getParameter("monto_recibido"), 0);
        con.setAutoCommit(false);
        try {
            // 1. Consecutivo de la factura: FAC-000001
            ps = con.prepareStatement(
                "SELECT CONCAT('FAC-', LPAD(COUNT(*) + 1, 6, '0')) AS num "
                + "FROM factura");
            rs = ps.executeQuery(); rs.next();
            numeroFactura = rs.getString("num");
            cerrar(rs, ps);

            // 2. Total real del pedido (nunca se confía en el formulario)
            ps = con.prepareStatement(
                "SELECT total, estado FROM pedido WHERE id_pedido = ?");
            ps.setInt(1, idPedido);
            rs = ps.executeQuery(); rs.next();
            total = rs.getDouble("total");
            estado = rs.getString("estado");
            cerrar(rs, ps);

            if (!"SERVIDO".equals(estado)) {
                throw new SQLException("El pedido no está en estado SERVIDO.");
            }
            if (recibido < total) {
                throw new SQLException("El dinero recibido es menor que el total.");
            }

            // 3. Inserta la factura (uq_factura_pedido garantiza 1 sola)
            ps = con.prepareStatement(
                "INSERT INTO factura (numero_factura, id_pedido, id_cajero, "
                + "metodo_pago, monto_total, monto_recibido) "
                + "VALUES (?, ?, ?, ?, ?, ?)");
            ps.setString(1, numeroFactura);
            ps.setInt(2, idPedido);
            ps.setInt(3, idUsuarioSesion);
            ps.setString(4, metodo);
            ps.setDouble(5, total);
            ps.setDouble(6, recibido);
            ps.executeUpdate();
            cerrar(ps);

            // 4. Cierra el pedido
            ps = con.prepareStatement(
                "UPDATE pedido SET estado = 'PAGADO', fecha_cierre = NOW() "
                + "WHERE id_pedido = ?");
            ps.setInt(1, idPedido);
            ps.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
} finally {
    try {
        con.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

```

        cerrar(ps);

        // 5. Libera la mesa
        ps = con.prepareStatement(
            "UPDATE mesa SET estado = 'LIBRE' WHERE id_mesa = "
            + "      (SELECT id_mesa FROM pedido WHERE id_pedido = ?)");
        ps.setInt(1, idPedido);
        ps.executeUpdate();
        cerrar(ps);

        // 6. Bitacora
        ps = con.prepareStatement(
            "INSERT INTO seguimiento_pedido (id_pedido, id_usuario, "
            + "      estado_anterior, estado_nuevo, comentario) "
            + "VALUES (?, ?, 'SERVIDO', 'PAGADO', ?)");
        ps.setInt(1, idPedido);
        ps.setInt(2, idUsuarioSesion);
        ps.setString(3, "Pago en " + metodo);
        ps.executeUpdate();
        cerrar(ps);

        con.commit();
        facturado = true;
        cambio = recibido - total;

    } catch (SQLException ex) {
        deshacer(con);
        error = ex.getMessage().contains("uq_factura_pedido")
            ? "Ese pedido ya tiene factura." : ex.getMessage();
    } finally {
        con.setAutoCommit(true);
    }
}

/* ----- B) Datos del pedido para mostrar en pantalla ----- */
ps = con.prepareStatement(
    "SELECT p.numero_pedido, p.estado, p.total, m.numero AS mesa "
    + "FROM pedido p LEFT JOIN mesa m ON m.id_mesa = p.id_mesa "
    + "WHERE p.id_pedido = ?");
ps.setInt(1, idPedido);
rs = ps.executeQuery();
if (rs.next()) {
    numeroPedido = rs.getString("numero_pedido");
    estado        = rs.getString("estado");
    total         = rs.getDouble("total");
    if (rs.getObject("mesa") != null) mesa = "Mesa " + rs.getInt("mesa");
}
cerrar(rs, ps);

} catch (SQLException ex) {
    error = ex.getMessage();
}

%>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

```

La transacción de la factura, paso a paso

Orden	Operación	Por qué está dentro de la transacción
1	Calcula el consecutivo FAC-000001.	Debe ser único: uq_factura_numero lo garantiza.
2	Relee el total y el estado del pedido en la base de datos.	El total nunca se toma del formulario; el navegador puede alterarlo.
3	Valida que el estado sea SERVIDO y que el dinero alcance.	Reglas del negocio antes de tocar nada.
4	INSERT en factura.	uq_factura_pedido impide una segunda factura para el mismo pedido.
5	UPDATE del pedido a PAGADO con la fecha de cierre.	Si fallara aquí, la factura existiría sin pedido cerrado: por eso va junto.
6	UPDATE de la mesa a LIBRE.	Deja la mesa disponible para el siguiente cliente.
7	INSERT en seguimiento_pedido.	Cierra la bitácora del pedido.

Las siete operaciones se confirman con un único commit(). Si cualquiera falla, el rollback devuelve todo al estado anterior: no queda una factura huérfana ni una mesa ocupada eternamente.

web/caja/facturar.jsp (parte 2: formulario y comprobante)

```
<div class="row justify-content-center">
<div class="col-lg-7">

<% if (error != null) { %>
<div class="alert alert-danger"><i class="bi bi-x-circle"></i> <%= esc(error) %></div>
<% } %>

<% if (facturado) { %>
<!-- ===== COMPROBANTE ===== -->
<div class="card shadow-sm">
<div class="card-body text-center">
<i class="bi bi-check-circle-fill text-success display-4"></i>
<h4 class="mt-2">Pago registrado</h4>
<p class="text-muted mb-4">Factura <b><%= numeroFactura %></b></p>

<table class="table table-sm w-75 mx-auto">
<tr><td class="text-start">Pedido</td>
<td class="text-end"><%= numeroPedido %></td></tr>
<tr><td class="text-start"><%= mesa %></td>
<td class="text-end">Cajero: <%= esc(nombreSesion) %></td></tr>
<tr><td class="text-start">Total</td>
<td class="text-end"><%= pesos(total) %></td></tr>
<tr><td class="text-start">Recibido</td>
<td class="text-end"><%= pesos(recibido) %></td></tr>
<tr class="fw-bold fs-5"><td class="text-start">Cambio</td>
<td class="text-end"><%= pesos(cambio) %></td></tr>
</table>

<div class="no-imprimir mt-3">
<button onclick="window.print()" class="btn btn-outline-dark">
<i class="bi bi-printer"></i> Imprimir</button>
<a href="<%= ctx %>/caja/cobrar.jsp" class="btn btn-success">
<i class="bi bi-arrow-left"></i> Volver a caja</a>
</div>
</div>
</div>

<% } else if ("SERVIDO".equals(estado)) { %>
<!-- ===== FORMULARIO DE PAGO ===== -->
<div class="card shadow-sm">
<div class="card-header bg-white fw-bold">
<i class="bi bi-receipt-cutoff"></i> Cobrar <%= numeroPedido %> (<%= mesa %>)
</div>
<div class="card-body">
<h2 class="text-center mb-4">Total: <%= pesos(total) %></h2>

<form method="post" action="<%= ctx %>/caja/facturar.jsp?id=<%= idPedido %>">
<div class="row g-3">
<div class="col-md-6">
<label class="form-label" for="metodo">Metodo de pago</label>
<select class="form-select" id="metodo" name="metodo_pago">
<option value="EFECTIVO">Efectivo</option>
<option value="TARJETA">Tarjeta</option>
<option value="TRANSFERENCIA">Transferencia</option>
<option value="MIXTO">Mixto</option>
</select>
</div>
<div class="col-md-6">
<label class="form-label" for="recibido">Dinero recibido</label>
<input type="number" step="100" min="<%= (long) total %>"
value="<%= (long) total %>" class="form-control"
id="recibido" name="monto_recibido" required>
</div>
</div>
<div class="d-grid mt-4">
<button class="btn btn-success btn-lg">
<i class="bi bi-cash"></i> Registrar pago</button>
</div>
</form>
</div>
</div>

<% } else { %>
<div class="alert alert-warning">
El pedido <%= numeroPedido %> esta en estado <b><%= estado %></b>
y no se puede facturar. Solo se cobran los pedidos SERVIDOS.
</div>
<a href="<%= ctx %>/caja/cobrar.jsp" class="btn btn-secondary">Volver</a>
<% } %>

</div>
```

```
</div>
<% cerrar(con); %>
<%@ include file="/WEB-INF/jspf/pie.jspf" %>
```

Elemento	Explicación
<code>request.getMethod()</code>	Distingue si la página se abrió (GET) o si se envió el formulario (POST).
<code>min="<%= (long) total %>"</code>	El navegador impide escribir un valor menor que el total; aun así se valida en el servidor.
<code>window.print()</code>	Abre el diálogo de impresión del navegador para entregar el comprobante.
<code>clase no-imprimir</code>	Regla de estilos.css que oculta los botones y el menú en el papel.

Paso 13. Módulo de administración

El administrador mantiene la carta: agrega productos, cambia precios y marca lo que se agotó. Es también el mejor lugar para mostrar cómo se traduce el error de una restricción UNIQUE a un mensaje entendible.

Paso 13.1 productos.jsp

```
web/admin/productos.jsp
<!--
admin/productos.jsp - Mantenimiento del menu: crear productos,
cambiar precio y activar/desactivar disponibilidad.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<% String tituloPagina = "Productos"; %>
<%@ include file="/WEB-INF/jspf/cabecera.jspf" %>

<h3 class="mb-3"><i class="bi bi-card-list"></i> Menu del restaurante</h3>

<% if (request.getParameter("msg") != null) { %>
<div class="alert alert-success"><%= esc(request.getParameter("msg")) %></div>
<% } %>
<% if (request.getParameter("err") != null) { %>
<div class="alert alert-danger"><%= esc(request.getParameter("err")) %></div>
<% } %>

<div class="row g-4">
<!-- ===== Formulario de alta ===== -->
<div class="col-lg-4">
<div class="card shadow-sm">
<div class="card-header bg-white fw-bold">Nuevo producto</div>
<div class="card-body">
<form method="post" action="<%= ctx %>/admin/guardar_producto.jsp">
<div class="mb-2">
<label class="form-label" for="codigo">Codigo (unico)</label>
<input class="form-control" id="codigo" name="codigo"
maxlength="15" required placeholder="HAM-03">
</div>
<div class="mb-2">
<label class="form-label" for="nombre">Nombre</label>
<input class="form-control" id="nombre" name="nombre"
maxlength="80" required>
</div>
<div class="mb-2">
<label class="form-label" for="categoria">Categoria</label>
<select class="form-select" id="categoria" name="id_categoria">
<%
Connection con = null; Statement st = null; ResultSet rs = null;
try {
con = abrirConexion();
st = con.createStatement();
rs = st.executeQuery("SELECT id_categoria, nombre FROM categoria "
+ "WHERE activo = 1 ORDER BY orden_menu");
while (rs.next()) {
<option value="<%= rs.getInt("id_categoria") %>">
<%= esc(rs.getString("nombre")) %></option>
<%
}
cerrar(rs, st);
} catch (SQLException ex) { out.println("<option>" + esc(ex.getMessage()) + "</option>"); }
%>
</select>
</div>
<div class="row g-2 mb-2">
<div class="col-6">
<label class="form-label" for="precio">Precio</label>
<input type="number" class="form-control" id="precio" name="precio"
min="100" step="100" required>
</div>
<div class="col-6">
<label class="form-label" for="tiempo">Minutos</label>
<input type="number" class="form-control" id="tiempo"
name="tiempo_preparacion" value="10" min="1" max="90">
</div>
</div>
<div class="mb-3">
<label class="form-label" for="desc">Descripcion</label>

```

```

        <textarea class="form-control" id="desc" name="descripcion"
            rows="2" maxlength="255"></textarea>
    </div>
    <button class="btn btn-primary w-100">
        <i class="bi bi-save"></i> Guardar producto</button>
    </form>
</div>
</div>
</div>

<!-- ===== Listado ===== -->
<div class="col-lg-8">
    <div class="card shadow-sm">
        <div class="table-responsive">
            <table class="table table-hover align-middle mb-0">
                <thead class="table-dark">
                    <tr><th>Codigo</th><th>Producto</th><th>Categoria</th>
                        <th class="text-end">Precio</th><th class="text-center">Estado</th></tr>
                </thead>
                <tbody>
<%
            try {
                st = con.createStatement();
                rs = st.executeQuery(
                    "SELECT p.id_producto, p.codigo, p.nombre, p.precio, p.disponible, "
                    + "    c.nombre AS categoria "
                    + "FROM producto p JOIN categoria c ON c.id_categoria = p.id_categoria "
                    + "ORDER BY c.orden_menu, p.nombre");
                while (rs.next()) {
                    boolean disp = rs.getBoolean("disponible");
%>
                    <tr>
                        <td><code><%= esc(rs.getString("codigo")) %></code></td>
                        <td><%= esc(rs.getString("nombre")) %></td>
                        <td class="small text-muted"><%= esc(rs.getString("categoria")) %></td>
                        <td class="text-end"><%= pesos(rs.getDouble("precio")) %></td>
                        <td class="text-center">
                            <form method="post" action="<%= ctx %>/admin/guardar_producto.jsp">
                                <input type="hidden" name="accion" value="disponibilidad">
                                <input type="hidden" name="id_producto"
                                    value="<%= rs.getInt("id_producto") %>">
                                <input type="hidden" name="valor" value="<%= disp ? 0 : 1 %>">
                                <button class="btn btn-sm btn-<%= disp ? "success" : "outline-danger" %>">
                                    <%= disp ? "Disponible" : "Agotado" %></button>
                            </form>
                        </td>
                    </tr>
                </tbody>
            } catch (SQLException ex) {
                out.println("<tr><td colspan='5' class='text-danger'>"
                    + esc(ex.getMessage()) + "</td></tr>");
            } finally { cerrar(rs, st, con); }
%>
                </tbody>
            </table>
        </div>
    </div>
</div>
</div>
</div>

<%@ include file="/WEB-INF/jspf/pie.jspf" %>

```

El botón de la última columna es un formulario diminuto que envía el valor contrario al actual: si el producto está disponible manda 0 y si está agotado manda 1. Cuando la cocina se queda sin queso, en dos clics el producto desaparece de la carta del mesero.

Paso 13.2 guardar_producto.jsp

```

web/admin/guardar_producto.jsp
<!--
admin/guardar_producto.jsp - Inserta un producto nuevo o cambia su
disponibilidad. Demuestra el manejo de la violacion de un UNIQUE.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ include file="/WEB-INF/jspf/conexion.jspf" %>
<%@ include file="/WEB-INF/jspf/utilidades.jspf" %>
<% String[] rolesPermitidos = {"ADMINISTRADOR"}; %>
<%@ include file="/WEB-INF/jspf/seguridad.jspf" %>
<%
    request.setCharacterEncoding("UTF-8");
    String accion = request.getParameter("accion");

```

```

String destino = ctx + "/admin/productos.jsp";

Connection con = null; PreparedStatement ps = null;
try {
    con = abrirConexion();

    if ("disponibilidad".equals(accion)) {
        ps = con.prepareStatement(
            "UPDATE producto SET disponible = ? WHERE id_producto = ?");
        ps.setInt(1, aEntero(request.getParameter("valor"), 1));
        ps.setInt(2, aEntero(request.getParameter("id_producto"), 0));
        ps.executeUpdate();
        destino += "?msg=" + java.net.URLEncoder.encode(
            "Disponibilidad actualizada.", "UTF-8");
    } else {
        ps = con.prepareStatement(
            "INSERT INTO producto (codigo, nombre, descripcion, precio, "
            + "id_categoria, tiempo_preparacion) "
            + "VALUES (?, ?, ?, ?, ?, ?)");
        ps.setString(1, request.getParameter("codigo").trim().toUpperCase());
        ps.setString(2, request.getParameter("nombre").trim());
        ps.setString(3, request.getParameter("descripcion"));
        ps.setDouble(4, aDoble(request.getParameter("precio"), 0));
        ps.setInt(5, aEntero(request.getParameter("id_categoria"), 1));
        ps.setInt(6, aEntero(request.getParameter("tiempo_preparacion"), 10));
        ps.executeUpdate();
        destino += "?msg=" + java.net.URLEncoder.encode(
            "Producto guardado correctamente.", "UTF-8");
    }

} catch (SQLIntegrityConstraintViolationException ex) {
    // Aqui se traduce la violacion de UNIQUE a un mensaje del negocio
    String m = ex.getMessage();
    String amigable = m.contains("uq_producto_codigo")
        ? "Ya existe un producto con ese codigo."
        : m.contains("uq_producto_nombre_cat")
        ? "Ya existe un producto con ese nombre en la misma categoria."
        : "Datos duplicados: " + m;
    destino += "?err=" + java.net.URLEncoder.encode(amigable, "UTF-8");
} catch (SQLException ex) {
    destino += "?err=" + java.net.URLEncoder.encode(ex.getMessage(), "UTF-8");
} finally {
    cerrar(ps, con);
}
response.sendRedirect(destino);
%>

```

Importante: Observe el catch de `SQLIntegrityConstraintViolationException`. Es la excepción específica que lanza JDBC cuando se viola una llave única o foránea. Capturarla aparte permite mostrar "Ya existe un producto con ese código" en lugar del mensaje del motor, que el usuario final no entendería. Este detalle suele valorarse en la evaluación.

Paso 14. Configuración final del proyecto

Paso 14.1 index.jsp

```
web/index.jsp
<!--
index.jsp - Pagina de entrada del proyecto.
Si hay sesion activa va al panel; si no, al formulario de acceso.
-->
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%
    if (session.getAttribute("idUserario") != null) {
        response.sendRedirect("inicio.jsp");
    } else {
        response.sendRedirect("login.jsp");
    }
%>
```

Paso 14.2 web.xml

```
web/WEB-INF/web.xml
<?xml version="1.0" encoding="UTF-8"?>
<!--
web.xml - Descriptor de despliegue.
Version 4.0 = Java EE 8 (Tomcat 9). Para Tomcat 10 u 11 cambie
el espacio de nombres a jakarta (ver el paso 14 del taller).
-->
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
    version="4.0">

    <display-name>RestauranteWeb</display-name>

    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>

    <!-- La sesion caduca a los 30 minutos de inactividad -->
    <session-config>
        <session-timeout>30</session-timeout>
    </session-config>

    <!-- Todas las paginas se leen y escriben en UTF-8 -->
    <jsp-config>
        <jsp-property-group>
            <url-pattern>*.jsp</url-pattern>
            <page-encoding>UTF-8</page-encoding>
            <trim-directive-whitespaces>true</trim-directive-whitespaces>
        </jsp-property-group>
    </jsp-config>

    <error-page>
        <error-code>404</error-code>
        <location>/login.jsp</location>
    </error-page>
</web-app>
```

Etiqueta	Efecto
welcome-file-list	Página que se abre al entrar a la aplicación sin indicar archivo.
session-timeout	Minutos de inactividad antes de cerrar la sesión.
jsp-property-group / page-encoding	Obliga a UTF-8 en todas las páginas: evita que las tildes salgan como signos raros.
trim-directive-whitespaces	Elimina las líneas en blanco que dejan los scriptlets en el HTML generado.
error-page 404	Si alguien escribe una dirección inexistente, se le lleva al login.

Paso 14.3 Si usa Tomcat 10 u 11

A partir de Tomcat 10 el paquete javax.servlet se llama jakarta.servlet. El código JSP de este taller no cambia, pero la cabecera de web.xml sí:

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
    https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
  version="5.0">
```

Importante: Si el proyecto arranca pero todas las páginas muestran un error 500 con la frase "jakarta.servlet.ServletException" o al revés "javax.servlet no existe", el problema es esta incompatibilidad de versiones, no su código.

Paso 15. Desplegar y probar la aplicación

Paso 15.1 Puesta en marcha

#	Acción	Resultado esperado
1	Iniciar el servicio de MySQL.	El servicio aparece en ejecución.
2	Ejecutar el script de la base de datos.	10 tablas y 5 usuarios creados.
3	Copiar el driver a WEB-INF/lib.	El .jar se ve dentro de la carpeta.
4	Ajustar DB_USUARIO y DB_CLAVE en conexion.jspf.	Coinciden con su instalación de MySQL.
5	Ejecutar el proyecto en Tomcat.	Se abre http://localhost:8080/RestauranteWeb/
6	Abrir prueba_conexion.jsp.	Aviso verde con el número de productos.
7	Borrar prueba_conexion.jsp.	La página de diagnóstico ya no queda publicada.

Paso 15.2 Guion de prueba completo

Haga el recorrido con dos navegadores abiertos al tiempo (por ejemplo, uno normal y otro en modo incógnito) para simular al mesero y a la cocina trabajando a la vez. Es la mejor evidencia para el informe.

#	Usuario	Acción	Qué debe ocurrir
1	mesero1	Iniciar sesión.	Entra al panel con las tarjetas de mesero y caja.
2	mesero1	Nuevo pedido en la Mesa 1.	Se genera el número PED-AAAAMMDD-001 y la mesa queda ocupada.
3	mesero1	Agregar 2 hamburguesas sencillas y 1 gaseosa.	El detalle muestra las dos líneas y el total se calcula solo.
4	mesero1	Agregar otra vez la hamburguesa.	No se crea una línea nueva: la cantidad sube a 3 (restricción UNIQUE).
5	mesero1	Intentar abrir otro pedido en la Mesa 1.	La Mesa 1 ya no aparece en la lista de mesas libres.
6	mesero1	Confirmar y enviar a cocina.	El pedido pasa a CONFIRMADO y ya no es editable.
7	cocina1	Abrir el tablero.	La comanda aparece con sus productos y el tiempo de espera.
8	cocina1	Iniciar preparación.	Estado EN_PREPARACION y queda registrado el cocinero.
9	cocina1	Marcar como listo.	Estado LISTO; el mesero lo ve de primero en su lista.
10	mesero1	Marcar como servido.	Estado SERVIDO y aparece el botón para ir a la caja.
11	cajero1	Facturar con efectivo.	Se genera FAC-000001, se calcula el cambio y la mesa vuelve a estar libre.
12	cajero1	Intentar facturar el mismo pedido otra vez.	Mensaje "Ese pedido ya tiene factura" (restricción uq_factura_pedido).
13	admin	Crear un producto con un código repetido.	Mensaje "Ya existe un producto con ese código".
14	cocina1	Abrir mesero/pedidos.jsp escribiendo la dirección.	Lo devuelve al panel con el aviso de permisos.

Paso 15.3 Comprobación en la base de datos

Después del recorrido, ejecute estas consultas para demostrar que la información quedó completa y trazable:

```
-- Bitacora completa del pedido: quien hizo que y a que hora
SELECT s.fecha_hora, s.estado_anterior, s.estado_nuevo,
       CONCAT(u.nombres, ' ', u.apellidos) AS responsable, s.comentario
FROM seguimiento_pedido s JOIN usuario u ON u.id_usuario = s.id_usuario
WHERE s.id_pedido = 1 ORDER BY s.fecha_hora;

-- Tiempos de servicio del día
SELECT numero_pedido,
       TIMESTAMPDIFF(MINUTE, fecha_confirmacion, fecha_listo) AS min_cocina,
       TIMESTAMPDIFF(MINUTE, fecha_creacion, fecha_cierre)   AS min_total
FROM pedido WHERE estado = 'PAGADO' AND DATE(fecha_cierre) = CURDATE();

-- Venta del día por categoria
```

```
SELECT c.nombre, SUM(d.cantidad) AS unidades, SUM(d.subtotal) AS venta
FROM pedido p JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
      JOIN producto pr ON pr.id_producto = d.id_producto
      JOIN categoria c ON c.id_categoria = pr.id_categoria
WHERE p.estado = 'PAGADO' AND DATE(p.fecha_cierre) = CURDATE()
GROUP BY c.id_categoria ORDER BY venta DESC;
```

16. Errores frecuentes y cómo resolverlos

Mensaje o síntoma	Causa	Solución
ClassNotFoundException: com.mysql.cj.jdbc.Driver	El driver no está en WEB-INF/lib.	Copie el .jar dentro de WEB-INF/lib y reinicie Tomcat.
Access denied for user 'root'@'localhost'	Usuario o clave equivocados.	Corrija DB_USUARIO y DB_CLAVE en conexion.jspf.
Unknown database 'restaurante_fastfood'	No se ejecutó el script.	Ejecute el .sql completo en MySQL Workbench.
Duplicate method abrirConexion()	conexion.jspf se incluyó dos veces en la misma página.	Recuerde que los fragmentos no se anidan: inclúyalo una sola vez por página.
Variable rolesPermitidos no encontrada	Se incluyó seguridad.jspf antes de declararla.	Declare el arreglo en la línea anterior al include.
response already committed / IllegalStateException	Se escribió HTML antes de un sendRedirect.	Ponga las redirecciones al comienzo de la página y use return; después.
Las tildes salen como caracteres raros	Falta la codificación UTF-8.	Revise pageEncoding, el meta charset, el web.xml y characterEncoding en la URL.
Los totales quedan en cero	Falta el procedimiento sp_recalcular_total.	Ejecute la sección 5 del script SQL.
The server time zone value is unrecognized	MySQL 8 sin zona horaria en la URL.	Agregue serverTimezone=America/Bogota a DB_URL.
La mesa no aparece al crear un pedido	Tiene un pedido abierto.	Es el comportamiento correcto: cierre o cancele el pedido anterior.
Table 'pedido' doesn't exist	Se está usando otro esquema.	Ejecute USE restaurante_fastfood; antes de las consultas.
El menú de Bootstrap no despliega	No se cargó el bundle de JavaScript.	Verifique el script de pie.jspf y la conexión a internet.

Anexo A. Inventario de archivos entregados

Archivo	Líneas	Contenido
sql/01_base_datos_restaurante.sql	294	Esquema completo, datos iniciales, procedimiento y usuarios.
web/WEB-INF/jspf/conexion.jspf	56	Conexión JDBC.
web/WEB-INF/jspf/utilidades.jspf	70	Cifrado, formatos y validaciones.
web/WEB-INF/jspf/seguridad.jspf	34	Control de sesión y rol.
web/WEB-INF/jspf/cabecera.jspf	74	Encabezado y barra de navegación.
web/WEB-INF/jspf/pie.jspf	13	Cierre y JavaScript de Bootstrap.
web/login.jsp	82	Formulario de acceso.
web/acceso.jsp	75	Validación de credenciales.
web/inicio.jsp	150	Panel con indicadores.
web/mesero/pedidos.jsp	98	Lista de pedidos.
web/mesero/nuevo.jsp	85	Apertura de pedido.
web/mesero/pedido.jsp	297	Carta y detalle.
web/mesero/acciones.jsp	264	Controlador del mesero.
web/cocina/tablero.jsp	141	Tablero de comandas.
web/cocina/accion.jsp	92	Cambios de estado en cocina.
web/caja/cobrar.jsp	71	Pedidos por cobrar.
web/caja/facturar.jsp	214	Factura y comprobante.

web/admin/productos.jsp	133	Administración del menú.
web/admin/guardar_producto.jsp	59	Alta de productos.
web/index.jsp, logout.jsp, prueba_conexion.jsp	-	Entrada, salida y diagnóstico.
web/css/estilos.css	17	Estilos propios.
web/WEB-INF/web.xml	37	Descriptor de despliegue.